Drishti दृष्टि

# Drishti on AWS: the lake, Delta or Iceberg

For platform and infrastructure teams: the lake in S3, the table format chosen per connector, the risk lake, bank scale, the scripts, the keys, the partitions and every optimisation

Delta is the default, Iceberg a full alternative: VaR, intraday and exposure read from either; official results to Delta or Iceberg, on disk or S3 · VaR sharded across servers and the parallel Java loader are built, on either format · only the super-scale scale-out of the whole lake is designed

AWS scripts for both formats tested with --help and --dry-run; Delta on S3, lake publish, the Iceberg engine and official results tested on S3 simulators; the first run in a real AWS account, and the Glue and S3 Tables runs, are still to do

*See what your data hasn't told you yet.*

ANY DATA. ANY DOMAIN. ONE GRAMMAR.

Ashutosh Sinha · October 2026

# Executive summary



| | In one line | Slides |
|---|---|---|
| **The lake lives in S3** | Every table's Parquet data and its log or metadata are in S3; EFS keeps only state, caches, the loader's scratch and locks. 200 TB costs about USD 1,400 a month[1] tiered in S3, USD 60,000 in EFS Standard. | 4 |
| **Delta or Iceberg, per connector** | Nothing above the storage plugin knows the format: pages, searches, history and BI give the same 18 answers; VaR and exposure are equal to the cent. Delta is the default; Iceberg is chosen by configuration. | 23 |
| **One load, one commit a day** | JSON Lines land in S3. Delta builds, syncs and publishes one conditional-PUT commit; Iceberg's loader writes S3 and commits itself. A reader sees the old day or the new, never a mixture. | 8 |
| **The risk lake** | VaR and exposure inputs as `list<double>` vectors, computed for any scope; official results append-only for 7 years, to Delta or Iceberg on `s3://`, JSON Lines beside. | 45 |
| **Bank scale** | 2.3M positions: about 7 GB of vectors a date[2], 12.4 TB over 7 years (ESTIMATED). VaR sharding and the parallel Java loader are built; the 200K benchmark is PLANNED. | 60 |
| **Status** | Every AWS script dry-run tested; S3 behaviour proven on three simulators; not yet run in a real AWS account; the Glue and S3 Tables runs NOT RUN. | 54 |
| **To decide** | Hot files on NVMe; the first run in a development account; the format per domain; the Iceberg catalog; the fixed gossip port; compaction on S3. | 94 |

[1] tier split × S3 list prices = USD 1,381; EFS: 200,000 GB × USD 0.30 [2] 2.3M × 509 × 8 B × 0.75 Parquet ratio; × 1,764 dates Full derivations: slides 121–123.

# What this deck covers




| | Chapter | What it holds | Slides |
|---|---|---|---|
| **01** | Why S3 | the one rule: the lake lives in S3, whatever the format; S3 against EFS at 200 TB | 4 |
| **02** | How it works | the flow, JSON Lines in, prefixes, partitions, files, one commit a day, safe commits, caches | 7 |
| **03** | Delta or Iceberg | where each plugs in, side by side, Iceberg as built, the decision guide, converting, running both | 23 |
| **04** | The risk lake | vectors, intraday and Kafka batches, official results to Delta or Iceberg, exposure paths | 45 |
| **05** | Evidence | status and tests, S3 simulators, Delta = Iceberg, bank scale, the loader, sharding, the demo lake | 53 |
| **06** | Operating it | setup at a glance, the daily load, maintenance, tiers, staying fast, sizing, cost, DR | 69 |
| **07** | What comes next | designed, proposed, limits, decisions | 90 |
| **A** | Appendix | setup steps 0–11, IAM, columns, the Iceberg spec, runbooks, scripts, derivations, glossary | 95 |
| | Related documents | the docs behind every slide | 125 |

# Why S3

01



# The one rule: the lake lives in Amazon S3



**Every table's Parquet data files AND its log (Delta `_delta_log/`) or metadata (Iceberg `metadata/`) are in S3. Nothing of the lake is on EFS. Delta or Iceberg is chosen per connector.**

EFS holds only what a node must find again after a restart: configuration, users and tokens, Drishti*net state, the content store's filesystem backend if chosen, and the caches that should be warm after a restart.

- **S3 (the lake):** Delta: `s3://<org>-drishti-<env>-lake/lake/<domain>/<table>/` with `business_date=…/PRODUCT_TYPE=…/part-….snappy.parquet` and `_delta_log/`; Iceberg: `…-lake/iceberg/<domain>/<table>/` with `metadata/` and `data/business_date=…/`
- **Domains:** trading, reference, legal, collateral, credit, market, macro, news, civic, **risk** (VaR and exposure inputs), **risk-results** (official results, append-only), us-equities, gleif, funds
- **S3 (landing):** the sources' dumps as JSON Lines, `<date>/<source>/*.jsonl.gz`, kept 30 days (`LANDING_KEEP_DAYS`)
- **S3 (logs):** access logs, SSE-S3, expire after 400 days
- **EFS (state):** `/drishti/<env>/<node>/opt/drishti/{data,config,sutras}`, one access point per node per path, identical path inside the container
- **EFS (loader scratch):** one day of one table for the minutes of a load, then removed; the per-table lock files
- **Never on EFS:** lake data, `_delta_log/`, `metadata/` (EFS is a staging place only: the loader's scratch, the Kafka stream's windows)

# Why S3 and not EFS: the comparison



| | Amazon S3 | Amazon EFS |
|---|---|---|
| **Cost of 200 TB (list prices, one Region)** | about **USD 1,400 a month**[1] tiered by business date; USD 4,600 all in Standard[2] | **USD 60,000 a month**[3] in EFS Standard; USD 3,200 in Infrequent Access[4] plus a charge on every byte read |
| **Request rate** | at least 3,500 writes and 5,500 reads a second per prefix; every `business_date=…/` folder is a prefix | one file system's limits, shared by every node, loader and maintenance job |
| **Latency of a single read** | 10–30 ms to the first byte in-Region; Drishti reads one row group | about 1 ms for hot data in Standard; tens of ms in IA |
| **Durability** | eleven nines, ≥ 3 AZs, versioning, Cross-Region Replication | Regional (three zones); backup by AWS Backup |
| **Tiers** | Standard → Standard-IA → Glacier Instant Retrieval (still ms), by date | Standard → IA → Archive by last access only |
| **Other readers** | Athena, Trino, Spark, EMR, Glue read the same Delta tables; Snowflake too for Iceberg (through a catalog) | only something that mounts the file system |
| **Writers** | conditional writes make concurrent commits safe (the native Iceberg engine probes for them) | a hard link on a POSIX file system (what Drishti uses on local disk) |
| **Iceberg only: file paths in metadata** | `s3://bucket/…` is the same for every task; Cross-Region copies read with `relocate` | the path depends on the mount; a moved folder is a moved table |

At 200 TB+ EFS costs 10–40×[5] more and shares one throughput budget, whatever the table format. Drishti reads one day's files, one row group of one file, by byte ranges. **The lake is in S3; EFS keeps the small, hot, per-node state.**

[1] tier split × S3 list prices = USD 1,381 [2] 200 TB × USD 23 a TB-month [3] 200,000 GB × USD 0.30 a GB-month (EFS Standard) [4] the doc's figure (implies USD 0.016 a GB-month; to be confirmed) [5] 60,000 ÷ 1,381 ≈ 43×; 60,000 ÷ 4,600 ≈ 13× Full derivations: slides 121–123.

# How it works

02

# The picture: the whole flow on one page



**Sources**

- ENDUR, CALYPSO, MUREX, reference, risk batch
- JSON Lines `.jsonl(.gz)`; CSV deprecated

**S3 · <org>-drishti-<env>-landing**

- `2026-10-20/endur/*.jsonl.gz`, …/calypso/, …/murex/
- kept `LANDING_KEEP_DAYS` (30), then expired

**EventBridge Scheduler**

- 22:30 New York, Monday to Friday
- runs the loader task; nightly maintain, daily report

**ECS task · loader · load-day.sh, per table, under the table's lock (EFS)**

- **Delta:** 1 sync landing → EFS scratch  2 `lake build` → scratch table (sorted by id, 1 MB row groups)  3 sync data files → S3  4 `lake publish` ONE commit
- **Iceberg:** `IcebergLoader` reads the dumps, sorts each day by id, writes Parquet + manifests to S3, commits `vN+1.metadata.json` (conditional PUT)

**S3 · <org>-drishti-<env>-lake (THE LAKE)**

- `lake/…/_delta_log/` · `iceberg/…/metadata/` + `business_date=…` Parquet
- domains: trading … market, **risk**, **risk-results**, public data
- versioning · SSE-KMS · VPC endpoint only · tiers by business date

**ECS services drishti-a, -b, -c (server + console)**

- `delta` and `iceberg` connectors, native engines, ranged GETs
- VaR and exposure engines read `risk/`; caches in heap and on disk
- Drishti*net: gossip port + HTTP/2 between task IPs

**ECS task · maintenance (nightly)**

- Delta: checkpoint, vacuum · Iceberg: expiry, orphans · retention, 7-year floor

**EFS (Regional, encrypted, Elastic)**

- state, caches, loader scratch, locks; NO lake data

# The picture, in words: each box



| Box | What it is | Key facts |
|---|---|---|
| **Sources** | ENDUR, CALYPSO, MUREX, reference systems, the risk batch | JSON Lines, gzipped or not; one folder per date and source; JSON arrays still read; CSV input deprecated |
| **Landing bucket** | `<org>-drishti-<env>-landing` | `<date>/<source>/…`; dumps expire after 30 days (`LANDING_KEEP_DAYS`); same protections as the lake |
| **Scheduler** | EventBridge Scheduler | `LOAD_SCHEDULE` 22:30 in `LOAD_SCHEDULE_ZONE` (New York), Mon–Fri; also -maintain and -report |
| **Loader task** | ECS task `drishti-<env>-loader` | **Delta:** sync → build → sync → publish → metrics, per table under its EFS lock; scratch always removed. **Iceberg:** `IcebergLoader` (the server's exec jar): sorts each day by id, files of 250,000 rows, one overwrite of the day's partition, no publish step |
| **The lake** | `<org>-drishti-<env>-lake` | Delta: Parquet + `_delta_log` under `lake/`; Iceberg: `metadata/` + `data/business_date=…/` under `iceberg/`, files named by absolute paths; versioning, SSE-KMS, VPC endpoint only |
| **Server nodes** | ECS services `drishti-a/-b/-c` | server + console per task; `delta` and `iceberg` connectors, native engines, one row group per entity; Iceberg refreshes every 10 s; VaR and exposure engines; Drishti*net |
| **Maintenance task** | ECS task, nightly (`MAINTAIN_SCHEDULE`) | `lake maintain` per table under its lock (Iceberg: `IcebergMaintenance`, expiry, orphan files ≥ 72 h); risk domains never below 7 years; then setup-lifecycle.sh |
| **Catalog (Iceberg, optional)** | a REST catalog: Glue's Iceberg endpoint, Polaris, Snowflake Open Catalog | commits by the catalog's requirements (409 retried); vended credentials; path-based tables need none. On AWS: NOT RUN |
| **EFS** | Regional, encrypted, Elastic throughput | `/drishti/<env>/<node>/opt/drishti/{data,config,sutras}`; loader scratch, spill and locks; never lake data, `_delta_log` or `metadata/` |

# Generate JSON Lines, then load it into Delta



**Source or generator**
- writes JSON Lines
- `trades-<date>.jsonl.gz`

→

**Landing / build folder**
- `<date>/<source>/*.jsonl.gz`
- a restatement: a second file

→

**lake build · load_rows.py**
- promoted and vector columns
- sorted by id; one commit a date

→

**Delta table**
- `lake/<domain>/<table>/`
- read by the Delta connector

| Form | One line is | Written by → read by |
|---|---|---|
| **Document** | one entity as its source holds it: `{"TRADE_ID": "1000015", "LEGS": [...]}` | make_trades, bulk_trades, pack samples → `lake build`, the file connector |
| **Loader row** | `{"domain", "kind", "id", "date", "doc", "columns"}` | `make_data.py --jsonl` → PostgreSQL, Cassandra, Iceberg … loaders |
| **Lake row** | `{"domain", "kind", "table", "id", "date", "doc"}` | `make_data.py --lake`, generators → `load_rows.py` |
| **Staged lake row** | `{"id", "doc", "columns"}` | lake build's staging, official-results publishers → Java `LakeWriter` |

**The rule:** one JSON object per line, UTF-8, `.jsonl` or `.jsonl.gz`. Generating and loading are separate steps; a loader never rewrites a pack's samples. The lake built from JSONL is byte-for-byte the lake built before.

# CSV in is deprecated; JSON Lines out beside every format



### Coming in

- **JSON Lines first**; JSON arrays and per-entity `.json` still read
- **CSV input deprecated**: the file connector still serves feed files, warns once per file, names them in Admin → Health
- `drishti.py convert trades.csv` → `trades.jsonl` (also JSON arrays, gzip, folders)

### Going out

- every export and download offers JSONL **beside** CSV, Excel, PDF, PNG
- server: `search/jsonl`, SQL `format=jsonl`, BI runs, statements, VaR and exposure captures
- console: one ↓ per panel, CSV or JSON Lines; CLI `--jsonl`

```
drishti.py convert trades.csv                                    # trades.jsonl, a line per row
drishti.py convert data/feeds/fixing/SOFR-HISTORY.csv --rows-document --out data/feeds/fixing.jsonl
drishti.py convert data/feeds/settlement                         # settlement.jsonl, the file name as "id"
```

**Not data, so not JSONL:** Delta and Iceberg Parquet, the BI store, the VaR and exposure hot files, Arrow between the server and Python, manifests. **The guard:** two drill tests check that every generator writes JSON Lines and every export entry point offers it.

# Prefixes and tables



| Choice | Why |
|---|---|
| **One table per source system and kind (`trade_endur`, `_calypso`, `_murex`, read as a union)** | each source loads, restates and fails on its own; each log a third the size |
| **Domain as the first folder** | matches the connector's domain; IAM and lifecycle rules can name a domain |
| **`risk/` one table per input kind; `risk-results/` written only by the publishers** | engines read `risk/`; no engine reads `risk-results/` (a guard test) |
| **Tables by year (`trade_endur/2026/`)** | DESIGNED (S2): until then one table holds every year |

```
s3://acme-drishti-prod-lake/
└── lake/                    DRISHTI_DELTA_ROOT
    ├── trading/
    │   ├── trade_endur/   _delta_log/
    │   │   └── business_date=2026-10-20/
    │   │       └── PRODUCT_TYPE=IRS_FIXFLOAT/part-….parquet
    │   ├── trade_calypso/ …
    │   └── trade_murex/ …
    ├── reference/ counterparty/ market/ …
    ├── risk/                the engines' inputs
    │   ├── pnl-vector/  scenario-pv/  fx-scenario/
    │   ├── scenario-set/  risk-hierarchy/  var-limit/
    │   └── portfolio/  simulation-run/  exposure-path/
    ├── risk-results/        append-only, never read back
    │   ├── var-official/  exposure-official/ …
    │   └── jsonl/<table>/<date>/
    └── us-equities/ gleif/ funds/   public data
```

# An Iceberg table in S3: metadata/ beside data/



```
s3://acme-drishti-prod-lake/
├── lake/                       Delta tables (DRISHTI_DELTA_ROOT)
└── iceberg/                    DRISHTI_ICEBERG_ROOT = s3://…/iceberg
    └── trading/                the domain (a REST catalog's namespace)
        ├── trade_calypso/      one table per source system and kind
        │   ├── metadata/
        │   │   ├── v12.metadata.json   schema, spec, sort order,
        │   │   │                       properties, snapshots
        │   │   ├── version-hint.text   path-based tables only
        │   │   ├── snap-…-….avro       a manifest list per snapshot
        │   │   └── …-m0.avro           manifests: files, partitions,
        │   │                           column bounds
        │   └── data/
        │       ├── business_date=2026-10-19/
        │       └── business_date=2026-10-20/
        │           ├── 2026-10-20-00000-<uuid>.parquet  ids A… – C…
        │           └── 2026-10-20-00001-<uuid>.parquet  ids C… – Z…
        └── trade_murex/ …
```

| Choice | Why |
|---|---|
| **One table per source system and kind** | each source loads, restates and fails on its own (`table.trade: trade_calypso`) |
| **Domain as the first folder** | the connector's `domain`; IAM and lifecycle can name it |
| **`metadata/` beside `data/`** | the whole table under one prefix: copy, replicate, grant as a unit |
| **Absolute paths in metadata** | a moved table needs `relocate` (native engine) |

# Partition keys and request-rate-friendly prefixes



| Key | Delta on AWS | Iceberg | Why |
|---|---|---|---|
| `business_date` | always the first key | the only partition (identity) | every interactive read names a date; a load and a restatement replace one day; retention removes whole days; tiers by the folder's date |
| `PRODUCT_TYPE` **(trades)** | second key (`--partition-by`) | not a partition | spreads a Delta day over several prefixes, keeps each partition sortable in one batch, lets a source restate one product type; Iceberg sorts a whole day externally |
| **nothing finer** | two keys for trades, one for the rest | `business_date` only | each partition column multiplies the folders and small files of a day |
| **risk tables** | `business_date` only | `business_date` only | the engine reads a whole date's vectors in one streamed pass; it sorts into hierarchy order in memory |
| **hidden transforms** | — | read for any table (`day(ts)`, `bucket(id)`) | other engines' tables, served by `iceberg-external` |

- S3 scales each prefix to ≥ 3,500 writes and 5,500 reads a second and splits busy prefixes by itself; a day's reads spread over its product-type folders, and over three tables for trades
- Drishti's writers name the folders, so the spread comes from the partition keys; Iceberg's hashed object-storage layout is not used (evaluate only if 503 Slow Down appears: PROPOSED)
- A 503 Slow Down is retried with back-off and counted in Health; alarm `drishti-<env>-lake-s3-5xx` fires on a sustained rate

# What each read needs from the table

| Need | What it reads | So the table needs |
|---|---|---|
| **One entity (`TRD MX-20000017`)** | the day's id map, then one row group of one file | rows sorted by id per file; small row groups |
| **History (`TRD MX-20000017 30`)** | the days whose id map holds it, one read per day, eight at once | `business_date` as the partition; nothing else |
| **Type-ahead** | the newest day's ids (and `index.days`), plus promoted name columns | `id`; `layout.<kind>.type-ahead` for names |
| **Search, pick lists, derived kinds, impact, links** | the day's column set: id and every promoted column, never `doc` | every field a search filters, sorts or shows promoted |
| **A date's vectors (VaR, exposure)** | one vector column and a few text columns, streamed file by file | `layout.<kind>.vectors`; whole dates, no deletion vectors |
| **File and row-group skipping** | Delta: per-file min/max in the log; Iceberg: manifest bounds per file; both: footer statistics per row group | Delta: `delta.dataSkippingStatsColumns` = id, business_date, promoted; Iceberg: metrics full on `id`, on promoted, none on `doc` |

# The numbers: Delta and Iceberg



| Setting | Delta on AWS | Iceberg | Why |
|---|---|---|---|
| **Rows per file** | `layout.<kind>.file-rows` (250,000) | `--file-rows` 250,000 (≈ 175 MB, estimate[1]) | few files per day, each large enough for S3 |
| **Row group** | 1 MB (`LAKE_ROW_GROUP_KB=1024`; writer default 256 KB) | 1 MB **compressed** (`--row-group-mb`): 300–700 trades | a single read fetches one row group: 1 MB balances bytes against 10–30 ms GETs |
| **Compression** | Snappy (the native engine also reads ZSTD, GZIP, LZ4_RAW) | zstd (about 9× on documents) | the writers' choices |
| **Statistics** | the log: id, business_date, promoted | manifests: `id` full, promoted, none on `doc` | file and row-group skipping without documents in the metadata |
| **Sort** | by id within a batch (`LAKE_BATCH_ROWS` 250,000) | by id over the whole day (external sort, spill) | no partition-size limit on Iceberg |
| **Commit unit** | one per table per day (`lake publish`) | one per day of ≥ 100,000 rows; small days together | fewer commits for small tables |
| **Checkpoint, metadata** | a checkpoint every 10 commits and after every maintenance run | 50 metadata versions kept, older deleted after a commit | a reader replays a short log, or reads one small pointer |
| **History kept** | log 30 days (`delta.logRetentionDuration`); removed files 7 days (`LAKE_VACUUM_HOURS` 168) | snapshots 168 h (`--expire-hours`) | readers in flight and *known at* reach back a week; **7 years for risk** on both |

[1] 250,000 rows × 0.7 KB a trade (zstd, measured, ICEBERG_CONNECTOR.md §3) = 175 MB

# Sorted by id: Delta in batches, Iceberg by the day



- A file holds a contiguous id range and each row group a smaller one: a read of one id skips every other row group by its statistics (Iceberg's manifests keep each file's min and max `id`)
- A table written elsewhere (Spark): `repartitionByRange(…, "id").sortWithinPartitions("id")`; for Iceberg also `WRITE ORDERED BY id`

### Delta: `lake build` sorts a partition in one batch

- each partition (date × product type) that fits one batch (`LAKE_BATCH_ROWS`, 250,000) is sorted; a larger one is written in the dumps' order and load-day.sh warns
- **remedy 1:** ask the source to export sorted by id: the JSON Lines order becomes the file's order, no memory cost (`bulk_trades.py` does so)
- **remedy 2:** partition finer (`PRODUCT_TYPE`, or a book or desk column)
- **remedy 3:** raise `LAKE_BATCH_ROWS` with the loader's memory: rows × document size × 3; 250,000 trades of 10 KB ≈ 8 GB[1]; the loader has 30 GB

### Iceberg: the loader sorts the whole day

- read JSON Lines from a file or stdin; dates after tomorrow refused
- sort per day in memory up to `--buffer-mb`, spilled runs beyond: the spill directory needs about the JSON's size; **no batch limit**
- merge the runs in id order into files of 250,000 rows
- ≈ 40 s for a day of 1,000,000 trades[2] on one thread, 4 files, 0.70–0.75 GB (ESTIMATED from a trial); 17,910 sample rows into 46 tables in 8 s, 30,000 trade-days in 5 s (MEASURED)

DESIGNED (SUPER_SCALE S1): range maps from the log's per-file id ranges instead of id maps, and Parquet Bloom filters on id.

[1] 250,000 rows × 10 KB a document × 3 (the doc's memory rule, AWS_DELTA_LAKE.md 7.2) = 7.5 GB ≈ 8 GB [2] one writer thread on a trial of 1,000,000 trades (ICEBERG_CONNECTOR.md §6); files 1,000,000 × 0.7 KB ≈ 0.70 GB in 4 files of 250,000 rows

# One commit per day per table

## Delta (`load-day.sh`)

**lake build**
- writes the day into a FRESH scratch table on EFS: every file there is live

→

**aws s3 sync**
- copies the data files to S3: new UUID names, nothing in S3 is overwritten

→

**lake publish**
- ONE commit that adds the new files and removes the day's previous files

## Iceberg (`IcebergLoader`)

**read**
- the day's JSON Lines

→

**sort**
- per day, spilled runs beyond `--buffer-mb`

→

**merge**
- runs in id order; files of 250,000 rows

→

**commit**
- ONE overwrite of the day's partition

- A reader sees **the old day or the new day, never a mixture**; the previous version stays readable (*known at*) until vacuum (`LAKE_VACUUM_HOURS`, 168 h) or snapshot expiry (`--expire-hours`, 168); 7 years for risk
- Delta: about **1,500 commits a year per table**[1] including maintenance; `lake publish` checks every file is in S3 (on the store it commits to) and refuses a change of columns
- Iceberg: a day of ≥ 100,000 rows is committed as soon as written, smaller days together; loading a day again replaces it

`lake build` and `lake demo` make one commit per business date (not per partition): sixty days of `trade_endur` went from 300 commits replaying the whole log (98 s) to 60. Intraday risk batches are the one exception: an appended commit per batch (slide 48).

[1] AWS_DELTA_LAKE.md 12.1's estimate: 252 load commits plus the nightly maintenance commits (retention, checkpoint, vacuum; Tue–Sat); not broken down in the doc

# Why concurrent commits on S3 are safe, in both formats



| Writer | How it commits |
|---|---|
| **`lake publish`, `lake maintain` (Python deltalake 1.x)** | conditional PUT of the commit file (`If-None-Match: *`): the second writer gets 412, re-reads the log and retries (20); no DynamoDB lock; unsafe rename refused |
| **Drishti's Java Delta writer (`lake build`, Kafka windows)** | local disk and EFS: a hard link of a finished commit file |
| **The official-results publisher** | on `s3://` each Delta commit file is an exclusive create (`If-None-Match: *`): two servers both land, the second on the next version |
| **Drishti's Iceberg writer, S3 path-based** | conditional PUT of `vN+1.metadata.json`, probed at the first commit; a store that ignores it makes the table **single-writer**; then `version-hint.text` |
| **Drishti's Iceberg writer, local disk or EFS** | the finished `vN+1.metadata.json` created under its final name by a hard link (fails if taken): 4 writers at once lose nothing |
| **Iceberg behind a REST catalog (Glue, Polaris)** | the catalog's commit with requirements (`assert-table-uuid`, `assert-ref-snapshot-id`, ...); a **409** is re-read, re-applied, retried |
| **The content store's s3 backend** | `If-None-Match: *` to create, `If-Match: <ETag>` to update; the loser gets `409 DRS-5057` |

**Two Iceberg writers on one table:** one wins each version; the other re-applies its change and retries (10). A re-applied rewrite of a day that changed meanwhile is refused; an append always re-applies; files of a commit that finally failed are deleted at once.

**Two layers of safety.** `check-conditional-writes.sh` proves the bucket refuses a second create; on top, the scripts take a lock per table on EFS (flock, NFSv4 byte-range locks, released if a task dies). **Tested:** three `lake publish` runs at once on moto, one winner per version; racing Iceberg writers on MinIO. No DynamoDB lock table in either format.

# The content store: cluster state on S3 without a shared disk



**One interface, four backends (config/store/store.yaml)**

- get · compare-and-set put · list · delete · changes

**filesystem**

- default; a shared mount (EFS) on a cluster

**sqlite**

- single node

**postgres**

- one SQL table

**s3**

- conditional PUT; ETag versions

- **s3:** `<prefix>/items/<key>`; create `If-None-Match: *`; update `If-Match: <ETag>`; a lost race is `409 DRS-5057`
- Two nodes saving one item at once: one success, one `409`; an unreachable store `503 DRS-5059`
- A dead node's BI run is adopted by exactly one node, by a compare-and-set lease

## What it holds on a cluster

- Designs; BI workspaces, items, versions, runs, schedules
- data-load overrides, connector edits, collaboration
- VaR and exposure captures and official records (append-only; row lists as JSONL)

**On ECS**

`s3` or `postgres` need no shared disk; `filesystem` needs EFS at the same path on every node. The s3 bucket needs the conditional-write check of step 4.

# What each server caches, Delta and Iceberg



| Cache | Where, bounded by | Keyed by (Delta · Iceberg) | After a restart |
|---|---|---|---|
| **file list / layout of each table** | heap: Delta from the last checkpoint; Iceberg per snapshot (`layout-cache` 64) | version · snapshot | re-read from the checkpoint or the metadata |
| **Parquet footers** | heap (native engines), 50,000 row groups, shared | the file (never changes) | re-read on first use (one ranged GET) |
| **manifests, delete files (Iceberg)** | heap: 2,000,000 manifest entries; 5,000,000 delete rows | the file | re-read on first use |
| **id maps of days** | heap (20 % shared) and disk (`id-map-disk.dir`, memory-mapped) | table version · **the day's fingerprint** | memory-mapped from disk: no scan |
| **column sets of days** | heap (`columns-cache-mb`; 1,024 on Iceberg) | as id maps | re-read (newest day in the background) |
| **documents** | heap and RocksDB on disk (`doc-disk-cache.dir`, 1 GB, 12 h) | as id maps | warm from disk |
| **history answers** | heap (`drishti.history.cache-mb`, 256 MB) | the kind's data version | rebuilt on ask |
| **VaR hot files** | `DRISHTI_VAR_HOT_DIR`: fast local disk; 2 dates open, 60 on disk | the date's inputs | rebuilt from the lake (deleted at shutdown by default) |
| **exposure hot files** | `DRISHTI_EXPOSURE_HOT_DIR`: local disk; dates within retention | the date's run | rebuilt from the lake |

**Iceberg's edge:** a commit of a new day leaves every other day's caches in place; on Delta a commit moves the table version, so untouched days reload (keeping them: PROPOSED). Disk caches and hot files default under `<data dir>`, on ECS on EFS.

# EFS or local NVMe for the disk caches and hot files



| | EFS (default) | Local NVMe (instance store) |
|---|---|---|
| **Latency** | ≈ 1 ms an operation (NFS); page cache serves repeated reads | tens of microseconds |
| **After a task moves host or zone** | warm: same folders | empty: rebuilt as users read |
| **Launch type** | Fargate or EC2 | EC2 only (m6id, r6id, i4i) |
| **Cost** | EFS Standard ≈ USD 0.30/GB-month[1] + Elastic charges | included in the instance |
| **Identical-path rule** | satisfied | not: the cache path is on the host |
| **Memory-mapped VaR and exposure hot files** | works, but each page fault is an NFS read | what the guides ask for: fast local disk |

```
# config/connectors/trade-endur.yaml
id-map-disk.dir:
  /opt/drishti/nvme/cache/trade-endur/idmaps
doc-disk-cache.dir:
  /opt/drishti/nvme/cache/trade-endur/docs
# the server task
DRISHTI_VAR_HOT_DIR=/opt/drishti/nvme/var/hot
DRISHTI_EXPOSURE_HOT_DIR=/opt/drishti/nvme/ccr/hot
```

**Recommendation**

Start the connector caches on EFS. Put the VaR and exposure hot files on NVMe from the start (`register.sh --nodes --cache nvme`, EC2): they are rebuildable and memory-mapped. The NVMe exception is the product owner's decision.

[1] EFS Standard list price, one Region, as quoted in AWS_DELTA_LAKE.md section 10

# Delta or Iceberg

03



# Where Delta and Iceberg plug in, and where they don't



Console and UI · terminals, pages, BI reports, Admin

Server · BI, VaR and exposure engines, captures, Drishti*net (replicas, shards, forwarding)

Packs and data contracts · kinds, promoted columns, JSON Lines in · connector names (trade-calypso)

Engine · SourceRouter: a kind to its connectors, in order; reads, searches, columns, history, scans

**Identical on both formats**

Nothing above the line knows whether a table is Delta or Iceberg. A pack routes kind `trade` to `trade-calypso`; the file under that name decides the format.

The seam · drishti-api `SourcePlugin`: fetch, days, columnar, columns, scan, dataVersion, heldDates, vectors, documents, tableIdentity …

**plugin: delta**

- Delta Kernel over `drishti-deltalake` (native) or Hadoop
- tables Drishti or others write
- vectors, whole-day documents

**plugin: iceberg**

- `drishti-iceberg` (native, default) or Apache Iceberg Java (library)
- Drishti's layout: id, doc, promoted
- loader and maintenance write

**plugin: iceberg-external**

- `drishti-iceberg` only
- anyone's tables, the row is the document
- read-only

**other plugins**

- files, S3 objects, PostgreSQL, DuckDB, Cassandra, Aerospike, MongoDB, Redis …

**drishti-lakeio** (shared): local disk, S3, byte ranges, codecs, footer cache, exclusive create, conditional PUT

**Format-specific below the seam only**

Storage · local disk or EFS (development, Windows) · Amazon S3 (`s3://`, `s3a://`) · Azure, GCS, HDFS only through the Hadoop/library engines

# What is identical, what differs, and why



| Layer | Identical on both | Differs | Why it differs |
|---|---|---|---|
| **Packs, contracts, connector names** | kinds, promoted columns (`layout.<kind>.columns`), JSON Lines input, routes | nothing | the format is below the plugin |
| **Logical table layout** | `id`, `doc`, `business_date`, promoted columns; sorted by id; one partition per day; 250,000 rows a file | row groups by rows (Delta, 1,000 rows or 1 MB on AWS) vs by compressed bytes (Iceberg, 1 MB) | Iceberg's writer sizes row groups in bytes |
| **Read paths** | id map, one row group per read, column sets, type-ahead, history, BI column scan | Iceberg caches by day fingerprint; Delta by table version | Iceberg's manifests give each day's files cheaply |
| **Where the table's truth is** | a folder in S3, Parquet data files | `_delta_log/` commits and checkpoints vs `metadata/` JSON, manifest lists, manifests | the two specifications |
| **Commits on S3** | a conditional PUT (`If-None-Match: *`): two writers never both win | Delta: `lake publish` (Python deltalake); Iceberg: the Java loader itself, plus a probe | Drishti's Java Delta writer writes local disk and EFS only |
| **Catalog** | none needed (path-based) | Iceberg may also use a REST catalog (Glue, Polaris) | Iceberg's ecosystem is catalog-centred |
| **Maintenance** | `lake maintain` and Admin → Lakes for both; retention, relayout, 7-year floor | Delta checkpoints and vacuum vs Iceberg snapshot expiry and orphan files | the two specifications keep history differently |

# Drishti's support, side by side



| Capability | Delta Lake (default) | Iceberg |
|---|---|---|
| **Documents (trades, reference, public data); local disk, EFS, S3 without Hadoop; Windows** | yes | yes |
| **Write a table in S3** | yes (`lake publish`) | yes (the loader, directly) |
| **Tables other engines write, in their own layout** | yes (Delta connector) | yes (`iceberg-external`, read-only) |
| **VaR inputs, intraday files, Kafka windows; exposure inputs (`exposure-path`)** | yes | yes (equal to the cent) |
| **VaR sharded across servers; the parallel Java loader (folder or S3)** | yes | yes |
| **Official results, folder or `s3://`; BI readers of them** | yes; BI follows the format by itself | yes; BI follows the format by itself |
| **Load and maintain** | `lake build`, `publish`, `maintain` | `lake build --format iceberg`, `maintain` |
| **Maintenance UI, schedules, ownership rule, 7-year risk floor** | yes | yes (+ orphan files) |
| **Disk caches (id maps, documents) across restarts** | yes | yes |
| **Caches survive a commit of another day** | PROPOSED | yes |
| **AWS scripts: load, backfill, maintain, lifecycle, alarms, report (dry-run tested)** | `deploy/aws/delta-lake/` | `deploy/aws/iceberg/` |
| **Catalog** | the table's log in S3 (none needed) | path-based in S3; REST, Glue, S3 Tables client (on AWS: NOT RUN) |
| **A promoted column added** | new table + backfill; in place: PLANNED | in place, by field id |
| **Conversion between the two (any table)** | PLANNED | PLANNED |

# The seam in detail: what each plugin implements today



| Capability (SourcePlugin method) | delta | iceberg | iceberg-external | Who needs it |
|---|---|---|---|---|
| **one entity, history days, search, reverse links (`fetch`, `days`, `search`, `reverse`)** | yes | yes | yes (no `days`) | every page |
| **columns of a day (`columnar`, `columns`), type-ahead index** | yes | yes | yes (columns) | searches, pick lists, impact |
| **BI column scan, data version, held dates, row estimate** | yes | yes | — | BI store and runs, statements |
| **time travel (*known at*)** | yes | yes | yes | every as-known-at view |
| **vector columns (`vectorColumns`, `vectors`): `list<double>` streamed** | yes | yes | — | VaR, exposure |
| **whole-day documents (`documents`)** | yes | yes | — | derived pages (faster path) |
| **table identity, holds none (`tableIdentity`, `holdsNone`)** | yes | yes | — | Drishti*net replicas vs shards |
| **inventory, refresh on signal (`inventory`, `refreshVersions`)** | yes | yes | — | Admin test, data-load signal |

**Clean layering, equal capability.** The gaps were implementations of existing interface methods, not changes to the architecture: the seam was already where it should be.

# Switching one table, and running both side by side

```
python tools/build_demo_lakes.py --store calypso=iceberg      # CALYPSO in Iceberg, ENDUR and MUREX in Delta
python tools/build_demo_lakes.py --store all=iceberg          # every system in Iceberg
```

## Before: every system in Delta

**trade-endur** · `plugin: delta` · `…/trade_endur`

**trade-calypso** · `plugin: delta` · `…/trade_calypso`

**trade-murex** · `plugin: delta` · `…/trade_murex`

## After: CALYPSO in Iceberg, the same connector name

**trade-endur** · `plugin: delta` · `delta/trading/trade_endur`

**trade-calypso** · `plugin: iceberg` · `iceberg/…/trade_calypso`, native

**trade-murex** · `plugin: delta` · `delta/trading/trade_murex`

| Compared, all-Delta against mixed (`stores_equality.py`) | Result |
|---|---|
| six trade pages; `TRD <id> 10` histories for each | the same |
| two counterparty-trades pages; three searches | the same |
| a BI model query | the same |
| total | **18 answers the same** |

**What stays:** the trading pack's route `trade → trade-endur, trade-calypso, trade-murex`, the kinds, the promoted columns, every page, search, history and BI model; BI column scan, data versions and held dates on both.

**What changes:** one connector file's `plugin`, `engine` and `table`.

**Never** two connectors for one system's trades: the union would double them.

# Inside each format: a log against a tree



## Delta Lake: a transaction log

**table folder**

- `s3://…/lake/trading/trade_endur/`

**_delta_log/**

- `…0451.json` `…0452.json` one commit each: add / remove + stats
- `…0450.checkpoint.parquet` every 10 commits
- `_last_checkpoint`: where to start replaying

**data files**

- `business_date=…/PRODUCT_TYPE=…/part-….snappy.parquet`

Read: replay checkpoint + later commits → the file list. Commit: create the next `NNN.json`.

## Iceberg: a metadata tree

**catalog pointer**

- `version-hint.text` (path) or the REST catalog's metadata location

**vN.metadata.json**

- schemas, partition specs, sort orders, snapshots, properties

**manifest list (per snapshot)**

- `snap-….avro`: the manifests, with partition summaries

**manifests**

- `…-m0.avro`: data and delete files, partition values, column bounds

**data and delete files**

- `data/business_date=…/…parquet`; position or equality delete files

Read: pointer → metadata → one manifest list → pruned manifests. Commit: create `vN+1.metadata.json` (or the catalog swaps the pointer).

# Side by side (1/3): log, commits, time travel, evolution

| Aspect | Delta Lake | Apache Iceberg | In Drishti today |
|---|---|---|---|
| **Where the truth is** | an ordered log of JSON commits + Parquet checkpoints | a tree: metadata JSON → manifest list → manifests | both read natively, no Hadoop |
| **Commit on S3** | create `_delta_log/N.json` if absent | create `vN+1.metadata.json` if absent, or a catalog swap | both by `If-None-Match: *`; Delta via `lake publish`, Iceberg via the Java loader |
| **Concurrency** | optimistic; conflicting writers retry | optimistic; requirements re-validated | a rewrite of a changed day is refused (Iceberg); per-table locks in the scripts (Delta) |
| **Time travel** | by version or timestamp, until vacuum and log cleanup | by snapshot or timestamp, until snapshots expire | *known at* on both; 7-year floor for risk tables on both |
| **Schema evolution** | add columns by name; renames need column mapping | by field id: add, drop, rename, reorder, widen | Iceberg reads every change by id; Delta publish refuses a changed day (in-place add: planned, not built) |
| **Reading cost of a refresh** | commits since the last checkpoint | one small pointer, then metadata if changed | Iceberg keys caches by day |

# Side by side (2/3): partitions, deletes, maintenance, catalogs



| Aspect | Delta Lake | Apache Iceberg | In Drishti today |
|---|---|---|---|
| **Partitioning** | Hive-style folders on columns; liquid clustering (Databricks) | **hidden**: transforms (day, bucket, truncate) in the spec; specs evolve | both: `business_date` only (+ PRODUCT_TYPE on Delta/AWS); Iceberg transforms read |
| **Row deletes, merges** | deletion vectors or file rewrites (copy-on-write) | position / equality delete files (merge-on-read) or rewrites | Drishti writes neither: a day is replaced; reads others' deletes (Delta: columns off; Iceberg: columns on) |
| **Maintenance** | OPTIMIZE, checkpoint, VACUUM | rewrite data and manifests, expire snapshots, remove orphans | both: `lake maintain` + Admin → Lakes; Iceberg with orphan removal |
| **Catalog** | optional (the log is the table); Unity, Glue for discovery | central to most engines (Glue, Polaris, Unity, Nessie); path tables work | Delta: none; Iceberg: path or REST, incl. Glue and S3 Tables (not yet run on AWS) |
| **Table identity across nodes** | table id in the log | table UUID in metadata | both report it to Drishti*net |

# Side by side (3/3): the ecosystem and the running cost



| Engine | Delta Lake | Apache Iceberg |
|---|---|---|
| **Apache Spark, Amazon EMR** | read and write (Delta's Spark library) | read and write (Iceberg's Spark runtime) |
| **Trino, Starburst** | Delta connector | Iceberg connector |
| **Amazon Athena** | reads Delta tables | reads and writes Iceberg tables in the Glue catalog; OPTIMIZE, VACUUM |
| **Snowflake** | reads through external tables / catalog integration | native Iceberg tables (managed or catalog-linked), Open Catalog (Polaris) |
| **Databricks** | native (Unity Catalog) | through Unity's Iceberg REST interface or UniForm |
| **pyiceberg, DuckDB, pandas** | delta-rs (deltalake), DuckDB's delta extension | pyiceberg, DuckDB's iceberg extension (both used in Drishti's tests) |

| Running cost item | Delta | Iceberg |
|---|---|---|
| **Storage (Parquet), tiers** | same | same |
| **Metadata per commit** | a small JSON; a checkpoint every 10 | a metadata JSON listing every snapshot kept + a manifest list + a manifest |
| **Catalog** | none | none (path) or Glue / S3 Tables request and storage charges |
| **Maintenance compute** | nightly `lake maintain` | nightly `IcebergMaintenance`, or managed optimizers (charged) |

# drishti-iceberg: Drishti's own Iceberg engine, no Hadoop

| | |
|---|---|
| IcebergSourcePlugin · IcebergExternalPlugin · IcebergLoader · IcebergMaintenance | unchanged read paths |
| catalog/ path-based (`version-hint.text`), a metadata file named directly, REST (OAuth2, SigV4, vended credentials) | Jackson, JDK HTTP |
| meta/ table metadata v1/v2, schemas, partition specs, sort orders, snapshots, refs; unknown fields kept | Jackson |
| manifest/ manifest lists and manifests: data and delete manifests, partition summaries, column bounds | Avro |
| scan/ + read/ pruning by partition and bounds; Parquet by field id; position and equality deletes | parquet-java |
| write/ Parquet with field ids and metrics, manifests, commits, maintenance operations | no Hadoop Configuration |
| drishti-lakeio local disk (Windows paths, UNC), S3 (AWS SDK v2), ranged reads, codecs, footer cache | shared with Delta |

## 0 Hadoop classes

loaded by connector, loader or maintenance (a test refuses them)

## Windows

no winutils.exe, no HADOOP_HOME: load-iceberg.ps1, start-server.ps1 -Iceberg

## The default engine

native; the library kept for Azure, GCS, HDFS

Refused by name, never read wrongly: format version 3, encrypted tables, ORC and Avro data files, unknown types, S3 remote signing.

# Two engines: native and library



| engine | What reads and writes | Reaches | Use it for |
|---|---|---|---|
| `native` **(default)** | drishti-iceberg, no Hadoop | local disk (Windows too), `s3://`, `s3a://`; path, named metadata, REST | everything on disk and S3; Windows |
| `library` | Apache Iceberg Java 1.10.1 over Hadoop | the above plus `abfs://`, `gs://`, `hdfs://` | Azure, GCS, HDFS; a fallback |
| `auto` | native on Windows, library elsewhere | | keeping the library on Linux |

| Measured (10,000 trades × 3 days) | native | library |
|---|---|---|
| **load (30,000 rows)** | 22.7 s | 25.7 s |
| **connector start (plan, type-ahead)** | 637 ms | 1,848 ms |
| **first trade (cold: id map, footer)** | 212 ms | 288 ms |
| **one trade, document cache emptied: median / p95** | 54 / 111 ms | 75 / 144 ms |
| **a trade on a past date (day not yet read)** | 94 ms | 353 ms |

**Why native is faster on first reads:** each Parquet footer, manifest and delete file is read once and kept (data files never change). The library re-reads the footer on every single-entity read.
Caches: footers 50,000 row groups (shared with Delta), manifests 2,000,000 entries, delete rows 5,000,000.

# Two connectors: Drishti's tables and anyone's tables

| | `iceberg` (Drishti's tables) | `iceberg-external` (anyone's tables) |
|---|---|---|
| **Written by** | `IcebergLoader` in Drishti's layout | Spark, Flink, Trino, Snowflake, Glue, dbt, pyiceberg, any layout |
| **The document** | the `doc` column, exactly as loaded | **the row** as JSON (structs, lists, maps), or a JSON column |
| **The id** | the `id` column | one or several columns; by default the table's **identifier fields** |
| **The business date** | `business_date` | a date column, a timestamp's day in a zone, the partition spec's date; or current only |
| **Searchable fields** | the promoted columns the pack declares | **every primitive column** (nested to depth 2) |
| **One entity** | id map: one row group | sorted or bucketed by id: one row group; unsorted: an **id index** built on first use per day |
| **Writes** | loader and maintenance | **never**: the owners' engines maintain the tables |
| **Deletes, time travel** | position and equality deletes applied; *known at* | the same; branches and tags not read |

# Catalogs: what Drishti supports today



| Catalog | How Drishti reaches it | Writes | Status |
|---|---|---|---|
| **Path-based (`catalog: hadoop`, the default)** | `<root>/<domain>/<table>/metadata/version-hint.text`; root a folder or `s3://` | loader and maintenance | |
| **A metadata file named directly** | `metadata-location.<kind>: s3://…/v12.metadata.json` | read-only | |
| **REST: Polaris, Snowflake Open Catalog, Nessie, Lakekeeper, Unity, Gravitino** | `catalog: rest`, `uri`, `warehouse`, OAuth2 `credential` or `token` | commits with requirements | |
| **AWS Glue Data Catalog, through Glue's Iceberg REST endpoint** | `uri: https://glue.<region>.amazonaws.com/iceberg`, SigV4 `signing-name: glue`; `check-glue.sh` | as REST | **NOT RUN** on AWS |
| **Amazon S3 Tables (table buckets)** | its Iceberg REST endpoint, SigV4 `signing-name: s3tables`; `check-glue.sh --s3-tables ARN` | as REST | **NOT RUN** on AWS |
| **The Glue and S3 Tables live runs** | `GlueLiveTest` with `DRISHTI_GLUE_LIVE=1`, in the product owner's account | — | **PLANNED** |
| **AWS Glue Data Catalog API (GetTable, UpdateTable); Hive metastore, JDBC** | not needed: Glue's REST endpoint; others behind a REST catalog | — | via REST only |

```
settings: { catalog: rest, uri: "https://glue.us-east-1.amazonaws.com/iceberg", warehouse: "<account-id>",
            catalog.rest.sigv4-enabled: "true", catalog.rest.signing-name: glue, catalog.rest.signing-region: us-east-1 }
```

# The catalog choice on AWS

## Path-based tables in S3

- no catalog service: `metadata/version-hint.text` in the table folder
- commits: conditional PUT, probe at first commit
- other engines read by naming the metadata file, or after registering the table in Glue
- **best when** only Drishti writes and Drishti reads

## AWS Glue Data Catalog (Iceberg REST endpoint)

NOT RUN

- `catalog: rest`, SigV4 `signing-name: glue`; `ICEBERG_CATALOG=glue` in the scripts (built; not yet run on AWS)
- commits through the catalog (409 retried); Athena, EMR, Redshift see the same tables
- IAM (and Lake Formation, if used) governs every table
- **best when** Athena or Spark also read or write

## Amazon S3 Tables (table buckets)

NOT RUN

- its Iceberg REST endpoint, SigV4 `signing-name: s3tables`, a table bucket ARN (written; not yet run on AWS)
- `check-glue.sh --s3-tables ARN`: the live run is the product owner's, not yet done
- AWS runs compaction and snapshot management itself
- **prove first** in a development account

**Recommendation (PROPOSED):** start path-based (built, no extra service, the same commit safety as Delta). Move a table behind Glue's REST endpoint when another engine must write it or Athena must query it; the connector's name, kinds and packs do not change, only `catalog`, `uri` and the signing settings. Run `check-glue.sh` first.

# Schema and partition evolution, as built

### Read by field id

renamed columns read correctly; added columns read as null; int→long, float→double, decimal widening; files without ids through `schema.name-mapping.default`

### Add a promoted column

the loader adds new promoted paths as columns; `IcebergMaintenance --columns trade:PATH` adds and fills them from the documents

### Partition evolution (read)

several specs in one table; every transform: identity, bucket, truncate, year, month, day, hour, void

### Hidden partitioning (read)

a table partitioned by `day(TRADE_TS)` or `bucket(id)` is pruned by its manifests; `iceberg-external` takes the business date from the spec

### Drishti's writer changes a spec

always `identity(business_date)`; no partition evolution written, none needed

**Versus Delta in Drishti:** adding a promoted column on Delta changes the table's columns, and `lake publish` then refuses the next day: a new folder (`trade_endur_v2`) and a backfill (Delta's in-place add: planned, not built).
On Iceberg the column is added in place by field id; old files read it as null until `--columns` rewrites them.

**Why it matters for a bank:** promoted columns grow as people search new fields. Iceberg absorbs that without a new table; Delta in Drishti's AWS flow needs a new table per change, because `lake publish` refuses a day whose columns differ.

# Snapshots, time travel and deletes

## A table's snapshot log, and what *known at* reads




A view *known at* that instant reads S3's schema and files; a later load is invisible; a time before the oldest snapshot kept reads nothing.

| | Iceberg in Drishti | Delta in Drishti |
|---|---|---|
| **Time travel unit** | a snapshot (each commit), from the snapshot log | a table version (each commit), from the log |
| **How far back** | until snapshots are expired: `--expire-hours` (168 by default) | until vacuum and log retention (168 h; 7 years in the risk domains) |
| **Rows deleted by other engines** | position and equality deletes applied by **every** read, columns stay on | deletion vectors honoured; the table's searches fall back to documents |
| **Drishti's own deletes** | none: a restated day replaces its partition | none: a restated day replaces its partition |
| **Format v3 deletion vectors** | refused by name (later) | — |

# Lake build to Iceberg, and the demo's CALYPSO in Iceberg



**1 JSON Lines in**

`data/testdata/<domain>/<kind>/*.jsonl.gz` or a bank's dumps

**2 lake build → Delta**

staged under `<data>/build/iceberg-stage`: the same documents and promoted columns as a Delta build

**3 IcebergLoader, engine native**

each business day committed as one overwrite of its partition; no Maven, works on Windows

**4 Iceberg table**

`<data>/iceberg/trading/trade_calypso` (or `s3://…`)

**5 connector file switched**

`trade-calypso.yaml`: `plugin: iceberg`, `engine: native`, `table: {trade: trade_calypso}`

```
tools/load-iceberg.sh ./data/iceberg --trades 10000 --days 3          # samples and a book, one load
tools/load-iceberg.sh s3a://risk-lake/iceberg --set s3.region=us-east-1 --spill-dir /data/tmp
python tools/lake/stores.py --from data/delta/trading/trade_calypso --root data/iceberg --table trade_calypso --replace
```

| Path to an Iceberg table | Scope |
|---|---|
| **`build_demo_lakes.py --store <system>=iceberg`** (via `stores.py`) | the trading domain's trade tables |
| **`load-iceberg.sh`** / **`load-iceberg.ps1`** → **`IcebergLoader`** | any kind, JSON Lines `{domain, kind, id, date, doc, columns}` |
| **`lake build --format iceberg`**; **`--store risk=iceberg`** (the whole risk domain) | any pack and kind; vector columns kept; `s3://` or a REST catalog |

# A decision guide for a bank



| If the bank ... | Choose | Because (Drishti today) |
|---|---|---|
| **has no format standard yet, or runs Kafka intraday heavily** | **Delta** (the default) | the default everywhere; small JSON commits; the longest-run AWS scripts and runbooks |
| **runs VaR or exposure and standardises on Iceberg** | **Iceberg** for the risk domain | `--store risk=iceberg`: VaR, exposure, intraday and official results equal Delta's to the cent |
| **already keeps trades in Iceberg (Snowflake, Glue, Spark pipelines)** | **Iceberg**, read in place | `iceberg-external` serves them read-only; no copy, no Drishti writer |
| **needs other engines to write the same tables Drishti reads** | **Iceberg** behind a REST catalog | commits through the catalog's requirements; Drishti's loader is one writer among several |
| **changes promoted columns often** | **Iceberg** for those tables | a column is added by field id; Delta's publish needs a new table |
| **standardises on Databricks and Unity Catalog** | **Delta** | Drishti reads Delta natively; Iceberg only through a REST catalog |
| **must run Drishti on Windows laptops for QA** | either | both native engines run without Hadoop |

# The recommendation, in one page

## Delta by default; Iceberg wherever the bank standardises on it

- **Risk domain** (P&L vectors, scenario PVs, exposure paths, official results): Delta by default; Iceberg equal to the cent
- **Trades, reference, market data**: either, chosen per source system (`--store`)
- **The bank's Iceberg tables** (Snowflake, Glue): `iceberg-external`, read in place
- **One switch per table**: the connector name never changes, so a table can move later

## Built for Iceberg; what remains

1. `vectors()`, documents, identity, disk caches; VaR, intraday, exposure on Iceberg
2. Official results to Iceberg and to `s3://` (both formats); BI readers follow
3. `deploy/aws/iceberg/`, orphan files, Admin → Lakes, the 7-year floor
4. VaR sharded across a cluster and the parallel Java loader, on either format
5. **NOT RUN:** the Glue and S3 Tables runs on a real AWS account (`check-glue.sh`)
6. **PLANNED:** Delta ↔ Iceberg conversion, Delta in-place schema evolution, the 200K benchmark

# Converting tables between formats



| Direction | How | History | Status |
|---|---|---|---|
| **Delta → Iceberg (trade tables)** | `stores.py` / `build_demo_lakes.py --store <system>=iceberg`: rows → `IcebergLoader` | current days; one snapshot per copied day | |
| **Risk domain, either way** | `build_demo_lakes.py --store risk=iceberg` (or `risk=delta`): rebuilt from JSON Lines | the days in the JSON Lines | |
| **Any table, Delta ↔ Iceberg** | a conversion of an existing table in either direction | current days | **PLANNED** |
| **In place: Delta UniForm, Iceberg `snapshot`/`migrate`** | other engines write the second format's metadata over the same Parquet | the converter's choice | **PROPOSED** (evaluate) |

**Keep the old table until the new one is proven:** build the Iceberg table beside the Delta one, run the equality test against both, then switch the connector file. Keep the Delta folder for its time travel (and the content store keeps every official capture), or archive it.
**Never** let two connectors serve one system's trades at once: the union would double them.

# Keeping one logical connector name across a switch



**1 Build beside**

write the Iceberg table next to the Delta one (`stores.py`, or the loader)

**2 Compare**

`stores_equality.py`-style answers, both lakes, the same dates

**3 Switch**

edit `trade-calypso.yaml`: `plugin: iceberg`, `engine: native`, `root`, `table.trade`, the same `layout`

**4 Signal**

the data-load signal, or wait `refresh-seconds`; Health shows `engine: native`, UP

**5 Keep, then retire**

keep the Delta table for its history; roll back by restoring the file

```
# config/connectors/trade-calypso.yaml (the name never changes)
plugin: iceberg                 # was: delta
kinds: [trade]
settings:
  domain: trading
  engine: native
  root: ${DRISHTI_ICEBERG_ROOT:${drishti.data.dir:./data}/iceberg}
  table: {trade: trade_calypso}
  layout: {trade: {columns: [TRADE_ID, SOURCE_SYSTEM, …]}}
```

**Nothing above the connector moves:** the pack's route, saved searches, BI models, reports and bookmarks name the connector or the kind, never the format.
Caches of the old connector are dropped when the file changes. At an upgrade the edited file keeps its values and gains the pack's new keys.

# The risk lake

04

# The risk lake's tables: inputs in risk/, results in risk-results/



| Table | One row is | Key | Vector column | Read by |
|---|---|---|---|---|
| **risk/pnl-vector** | a trade's (or position's) P&L over 509 scenarios | PNL-<trade>-<set> | PNL | VaR engine |
| **risk/scenario-pv** | a trade's scenario PVs and base PV, its own currency | SCENARIO_PV_ID | SCENARIO_PV | VaR engine |
| **risk/fx-scenario** | a currency's rate today and per scenario | FX_SCENARIO_ID | none (RATES in doc) | VaR engine |
| **risk/scenario-set** · **risk-hierarchy** · **portfolio** | the scenario days · a tree node with its limit · a portfolio | SCENARIO_SET_ID … | none | VaR engine |
| **risk/var-limit** | a node's VaR limit in a hierarchy by columns (the limit letters) | VL-<NODE_ID> | none | VaR engine |
| **risk/simulation-run** | an exposure run's grid and identity | RUN_ID | none | exposure engine |
| **risk/exposure-path** | a trade × grid date: MtM over the P paths | PATH_ROW_ID | MTM | exposure engine |
| **risk-results/var-official** | a node × date × result version | node@date#v | none | BI only (VOFF) |
| **risk-results/exposure-official(-cpty)** | a node, set, counterparty or group × version | scope@date#v | none (profile as JSON) | BI only (XOFC) |

Engines read risk/ only; nothing reads risk-results/ back except BI's read-only connectors (a guard test). Every input row carries LIMIT_LETTER. Positions are derived, never stored.

# A vector is one list<double> column: why, and how it is carried



| Layout (one date, 2M rows × 509) | Size, compressed (estimate) | Sum a scope's vectors | Verdict |
|---|---|---|---|
| **A. Long: a row per position × scenario** | ~9–11 GB[1] | a group-by over hundreds of millions of rows | rejected |
| **B. Wide: 509 columns** | ~5.5–6.5 GB | 509 column chunks per row group | rejected (schema = scenario count) |
| **C. Vector: `PNL list<double>`** | ~5.5–6.5 GB | one column, decoded into one array, added | **chosen, built** |
| **D. Blob: 4,072 bytes** | ~6–7 GB | one column, copy bytes | rejected (other tools cannot read it) |

### As built

- `layout.<kind>.vectors: [PNL]`; `lake build --vector`; both writers
- the document keeps `{"$vector": "PNL"}`: never stored twice
- streamed per date, one buffer reused for every row
- round trip JSONL → Delta → `double[]` equal to the bit
- `list<float>` read and written for exposure paths

### Where it runs

- streamed reads: the Delta plugin (default) and the Iceberg plugin (`SourcePlugin.vectors`)
- VaR and exposure on Iceberg: equal to Delta to the cent
- Iceberg loader: `list<double>`; a missing element null, never 0; a non-number list a reject
- bank-scale layout (sorted by legal entity and desk, BSS + ZSTD): the parallel Java loader writes it

[1] design estimates of VAR_GUIDE.md 29.1 for 2,000,000 rows × 509 × 8 B = 8.14 GB raw: vector and wide ≈ 0.68–0.8 × raw compressed; long adds an id and scenario index per value; ESTIMATED, not measured

# Morning run, intraday batches, Kafka micro-batches: all into the lake



### Morning run · RUN_KIND FULL

- every live trade, one BATCH_ID
- JSONL → `lake build` replaces the date

### Intraday files · INTRADAY

- changed trades only, own BATCH_ID
- `drishti.py var intraday --load` (append)

### Kafka PV topic · micro-batches

- `pv-updates` (var-pv-stream), JSONL text
- window 5 min · flush at 500 · Apply now

### risk/ in Delta or Iceberg (date D)

- the morning commit replaces D
- each intraday batch: one appended commit
- each Kafka window: ONE batch, `K-<date>-<time>-<n>`
- latest `COMPUTED_AT` per trade wins
- the lake is the record: a restart rebuilds
- 7 years of time travel

signal

### VaR engine (one date hot)

- full run: date reloads
- batch: new − old into the book and every node above
- official capture from the morning run only

### risk-results/var-official

- append-only, a row per node and version
- JSONL beside it
- a folder or `s3://`
- read by BI as `VOFF`

Measured: 200 batches of 3 trades[1] equal one full load to the cent, on every node; batch time does not grow.

[1] measured: VarIntradayApiTest (VAR_GUIDE.md 8.4)

# On AWS: who writes the risk lake, and where



| Table group | Written by | On an S3 lake today |
|---|---|---|
| **`risk/` morning runs (files)** | the bank's batch: JSONL → `lake build` → `lake publish` (overwrite-dates) | load-day.sh, like any table |
| **`risk/` intraday files** | `lake build --mode append` → `lake publish --mode append` | the tools; no script yet |
| **`risk/` Kafka windows** | the server's Java lake writer (`pv-updates' lake`) | local or EFS lake path |
| **`risk-results/` official** | the server's publisher, Delta (default) or Iceberg, + JSONL | written to `s3://` itself |

**Risk tables in S3 keep 7 years:** publish them with `--log-retention-days 2557 --deleted-file-retention-days 2557`; `lake publish`'s defaults (30 and 7 days) would cut time travel.

### Official results to S3

- `drishti.var.official.root: s3://<lake>/risk` (and the exposure root); `format: delta` or `iceberg`
- each Delta commit an exclusive create: two servers both land
- BI's `var-results` follows the format; task role: `s3:PutObject`, no delete

### Kafka windows to S3

PROPOSED

- `pv-updates`' lake on EFS; each window published with `--mode append`
- or keep `risk/` on EFS if intraday must be seconds

Hot files: `DRISHTI_VAR_HOT_DIR`, `DRISHTI_EXPOSURE_HOT_DIR` on fast local disk, as the guides say (slide 22).

# Official results: append-only, versioned, reproducible



| Column group | Content |
|---|---|
| **`business_date` · `id` · `doc`** | partition; `<NODE_ID>@<date>#v<RESULT_VERSION>`; the row as JSON |
| **node** | `HIERARCHY` (MGMT, LEGAL, LL …), `NODE_ID`, `NODE_NAME`, `PARENT_ID`, `LEVEL` |
| **version and inputs** | `RESULT_VERSION` (1, 2 … on restatement), `CAPTURE_ID`, `INPUT_VERSIONS` (versions, batch ids, rules digest) |
| **figures** | `VAR`, `ES`, `CONFIDENCE`, `ES_CONFIDENCE`, `STANDALONE_SUM`, `DIVERSIFICATION`, `HYPO_PNL` |
| **limits and backtest** | `LIMIT`, `LIMIT_USE`, `EXCEPTIONS_250`, `TRAFFIC_LIGHT`, `TRADE_COUNT`, `POSITION_COUNT` |

- every configured hierarchy: 131 nodes on the test data (MGMT, LEGAL + 43 limit-letter nodes)
- `delta.appendOnly`: nothing is overwritten; a restated date adds rows with the next `RESULT_VERSION`
- 7 years of log and removed files: time travel to any published day
- no vectors, no trade rows: the capture in the content store keeps the vector for audit
- Delta or Iceberg, a folder or `s3://`

**For other systems, never for engines.** BI reads the published tables as `VOFF` and `XOFC` (the reports Market risk: VaR and Counterparty risk; no measure sums VaR). Each figure can be recomputed from its inputs, the same vectors and engine version, years later.

# Official results: Delta or Iceberg, a folder or S3, JSON Lines beside



```
<root>/risk-results/                          a folder or s3://
├── var-official/              business_date=…/   Delta (default) or Iceberg, append-only, 7 years
├── exposure-official/  exposure-official-cpty/   append-only
└── jsonl/<table>/<date>/<table>-v<table version>.jsonl      one file per commit
```

| Setting | Default | Effect |
|---|---|---|
| **drishti.var.official.root** | ${DRISHTI_DELTA_ROOT} | where both are written: a folder or s3:// |
| **drishti.var.official.format** | delta | or iceberg; a table keeps the format it was created in |
| **drishti.ccr.exposure.official.format** | delta | the same for exposure |
| **….official.storage** | — | the s3.* settings (no keys: the task role) |
| **drishti.var.official.jsonl** | true | each VaR publish also written as JSONL beside the table |
| **drishti.ccr.exposure.official.jsonl** | true | each exposure publish also written as JSONL |

- **Append-only on both:** delta.appendOnly; Iceberg append commits only (drishti.append-only=true), a snapshot per run
- **7 years on both:** Delta log retention; Iceberg history.expire.max-snapshot-age-ms, every metadata file kept
- **On S3:** each Delta commit an exclusive create: two servers publishing at once both land
- **BI's readers** (VOFF, XOFC) switch to plugin: iceberg by themselves

- A restatement is a new file, never a rewrite; content-store row lists are JSONL too (var/official/<date>-membership-v<n>.jsonl); older .json records are read as they are
- Tested: OfficialSinkTest, OfficialS3Test (S3Mock), OfficialReadersTest; not yet run against a real bucket

# Exposure paths: the large table, and what is kept for 7 years



| Item (1,000,000 trades) | P = 2,000, G = 50 | P = 5,000, G = 100 |
|---|---|---|
| **trade paths, float32, dense, a date** | 400 GB[1] | 2.0 TB |
| **the same, float64** | 800 GB | 4.0 TB |
| **netting-set V matrices (hot), 100,000 sets** | 40 GB | 200 GB |
| **for comparison: VaR vectors, a date** | 4.07 GB | |

| What | Kept (decided 2026-10-26, built) |
|---|---|
| **trade paths** | 20 business days, and each month-end for 1 year |
| **netting-set matrices** | every date for 1 year; month-ends for 7 years |
| **statistics per set and counterparty** | every date, 7 years |
| **trade EE contributions** | every date, 7 years |
| **official captures, run records** | 7 years |

**3.5 PB → about 130 TB[2]:** 7 years of dense trade paths at P = 5,000, G = 100 would be 3.5 PB; under the decided tiers the standing storage is about 130 TB at full size (a fifth at P = 2,000, G = 50). ESTIMATED from the guide's formulas.

**Exposure-path layout at scale (EXPOSURE_GUIDE.md 15.3)** PROPOSED

- partition by `business_date` and `run_id`; sort by `NETTING_SET`, `TRADE_ID`, `GRID_INDEX`; ~512 MB files; `MTM` BYTE_STREAM_SPLIT + ZSTD

[1] trades × P × G × 4 B (EXPOSURE_GUIDE.md 27.2) [2] 2.0 TB × 1,764 dates; kept tiers summed = 132 TB Full derivations: slides 121–123.

# Evidence

05

# Status: what exists today, and how it was tested



No label: built, in the code and tested: the AWS lake, the risk lake, JSON Lines, the S3 work, official results to S3, Iceberg as an alternative, Drishti*net, VaR sharding and the parallel Java loader.

**DESIGNED** — The super-scale scale-out of the whole lake (SUPER_SCALE S1–S8), not built.

**PLANNED** — In the plan, not built: Delta in-place schema evolution, Delta ↔ Iceberg conversion, a bank-scale benchmark on both formats, Unity Catalog.

**NOT RUN** — Code written, never run against AWS: the Glue and S3 Tables live runs.

**PROPOSED** — This deck's recommendation, not yet in the docs (each one is marked where it appears).

**MEASURED** — A number measured on Drishti (the doc names the test or benchmark).

**ESTIMATED** — A number from arithmetic or a linear extrapolation of measured points; to be replaced by measurement.

### How it was tested

Every AWS script, Delta and Iceberg, with --help and --dry-run (`tools/test_deploy_aws.py`, an aws that fails if called). Delta on S3 and `lake publish` on moto, three publishes at once; official results to S3 on S3Mock in both formats (`OfficialS3Test`); VaR and exposure equal on Delta and Iceberg to the cent (`VarFormatEqualityTest`, `ExposureFormatEqualityTest`); sharded VaR equal to one server (`VarShardClusterTest`); the connector upgrade (`ConnectorUpgradeTest`).

**Not yet: the first run in a real AWS account (steps 0–11 and the runbooks); the Glue and S3 Tables live runs. Every number marked ESTIMATED waits for it.**

# Built (1/2): the lake on S3



| Capability | Where |
|---|---|
| **Reading Delta tables in S3: file lists from the last checkpoint, id maps, one row group per read, footers cached, documents cached on disk** | drishti-deltalake, drishti-lakeio, plugins/drishti-plugin-delta |
| **`lake build`: JSON Lines into a Delta table on local disk or EFS: promoted and vector columns, partitions, sorted by id per batch** | tools/lake/build.py, the Java writer in the server jar |
| **`lake publish`: a built table committed into an S3 table as ONE commit (S3 conditional writes)** | tools/lake/publish.py, test_publish.py |
| **`lake maintain` on `s3://`: retention, checkpoint, vacuum; relayout of the per-source trade tables** | tools/lake/maintain.py (`--force-foreign`) |
| **Buckets, IAM, conditional-write check, lifecycle tiers, alarms, report, load, backfill, maintenance, EFS, Cloud Map, task definitions, schedules** | deploy/aws/ (dry-run tested) |
| **ECS images (server, console, loader) and entry points** | deploy/docker/ecs/ (files) |
| **Drishti*net shards by table; replicas send one entity to the same node by a hash of its id** | DRISHTI_NET.md §7, §10 |

# Built (2/2): history, risk, Iceberg, sharding, the loader



| Capability | Where |
|---|---|
| History queries over the lake (`TRD <id> 30`, `knownAt`), the Delta fast path, across Drishti*net | HISTORY_QUERIES.md, DELTA_CONNECTOR.md §8 |
| Delta and Iceberg side by side, per source system; Drishti*net gossip on its own port, peer data over HTTP/2 | build_demo_lakes.py `--store`; NET.md §4, §7 |
| Content store with an S3 backend (conditional PUT, ETag); S3 on S3Mock, moto, MinIO | CONTENT_STORE.md, TOOLS.md §4.12 |
| Vector columns `list<double>`; VaR and `exposure-path` inputs in `risk/`; 7-year floor; official results append-only | VAR_GUIDE.md §15, §19, §29; EXPOSURE_GUIDE.md §15 |
| JSON Lines standard; CSV deprecated; official results as JSONL; scenario PVs; intraday and Kafka batches; DATAMAP | DATA_FORMATS.md, VAR_GUIDE.md §8.4 |
| Official results straight to `s3://`, Delta (default) or Iceberg; each Delta commit an exclusive create | VAR_GUIDE.md §19, AWS_DEPLOYMENT.md §22 |
| Iceberg a full alternative: VaR, intraday, exposure, official; `deploy/aws/iceberg/` | ICEBERG_CONNECTOR.md §18–19 |
| Connector upgrades: unedited files follow the pack, edited ones gain keys; hierarchies by columns, `var-limit` | CONNECTOR_FILES.md, VAR_GUIDE.md §5.2 |
| VaR sharded across Drishti*net: shards of legal entity and desk ranges, two owners each, partial vectors added | VAR_SHARDING.md |
| The parallel Java loader: JSON Lines (folder or S3) into Delta or Iceberg, one commit per date, shard-aligned files | PARALLEL_LOADER.md |

# Iceberg as built, and where each piece is tested



| Piece | Where, and how it is tested |
|---|---|
| **drishti-iceberg, no Hadoop: metadata, manifests, planning with pruning, field-id reads, deletes, time travel; the writer (Parquet with field ids and metrics, manifests, commits)** | `drishti-iceberg`, `drishti-lakeio`; pyiceberg interop both ways; a class loader that refuses Hadoop; `NativeCommitTest` |
| **`iceberg` connector: id maps, column sets, type-ahead, history days, BI scan, data versions, *known at*; `vectors()`, `documents()`, `tableIdentity()`, `inventory()`, `refreshVersions()`; disk caches** | `plugins/drishti-plugin-iceberg`; every plugin test on both engines; `IcebergParityTest`, `IcebergVectorsTest`, `IcebergDiskCacheTest` |
| **`iceberg-external`: anyone's tables, read-only; rows as documents, identifier fields, every column searchable, id index, `tables: auto`** | `IcebergExternalPluginTest` on pyiceberg-written fixtures |
| **Iceberg tables in S3 via the AWS SDK: the loader writes, the connector reads, commits by conditional PUT; TLS, private CA and proxy for the object store** | S3Mock (in-process and Docker), MinIO; racing writers on MinIO only; never real AWS S3 |
| **REST catalogs (OAuth2, ETag, vended credentials), incl. Glue's Iceberg endpoint and S3 Tables (SigV4)** | REST protocol tests; `SigV4Test`; `GlueLiveTest` written, NOT RUN on AWS |
| **Mixed storage per source system (`--store calypso=iceberg`)** | `tools/lake/stores.py`; equality test: 18 answers the same |
| **`IcebergLoader`, `IcebergMaintenance` on both engines; `load-iceberg.sh` and `.ps1` (Windows); orphan files, Admin → Lakes, the 7-year floor** | `IcebergOrphanFilesTest`, `test_maintain_iceberg.py` |
| **`deploy/aws/iceberg/`: load, backfill, maintain, lifecycle, alarms, report, IAM, Glue check; with the shared ECS images, EFS access points, Cloud Map, IAM roles and buckets** | `--help` and `--dry-run` in `test_deploy_aws.py`; first real run pending |
| **VaR, intraday and exposure from Iceberg; official results to Delta or Iceberg, disk or `s3://`** | `VarFormatEqualityTest`, `ExposureFormatEqualityTest`: Delta = Iceberg to the cent; `LakeAppendsTest`; `OfficialS3Test` |

# S3 on three simulators, no Docker: what each test proves



| Test | Simulator | What it proves | Runs on |
|---|---|---|---|
| `NativeOnS3SimulatorTest` | Adobe S3Mock 4.12.4 in the JVM; MinIO binary | the Iceberg loader writes, the connector reads a table in S3; writers at once where atomic | Linux, Windows (S3Mock) |
| `NativeOnS3Test` | S3Mock image; MinIO image (Testcontainers) | the same; the commit probe decides single-writer | where Docker is reachable |
| `S3MockContentStoreTest` | S3Mock in the JVM | the content store on S3 | Linux, Windows |
| `tools/lake/test_s3_simulators.py` | moto 5.2.3 (pure Python); MinIO binary | **Delta** on S3; `lake publish`: new table, a day restated, a missing file refused, three at once | Linux, Windows (moto) |
| `OfficialS3Test` | S3Mock in the JVM | official results to `s3://` in Delta and Iceberg; two writers in sequence; the exclusive create | Linux, Windows |
| `check-conditional-writes.sh` (`delta-lake/`, `iceberg/`) | your real bucket | the bucket refuses a second create (412): the probe both formats rely on | AWS (first run pending) |

**Conditional writes honoured:** S3Mock: create-only, update-if-match, delete-if-match, racing creates not atomic · moto: all three, racing creates one winner · MinIO: all, the most faithful (binary needed; skipped with the reason). Each suite probes and prints its line first.

```
./mvnw -q -pl drishti-identity,plugins/drishti-plugin-iceberg -am test -Dtest='S3MockContentStoreTest,NativeOnS3SimulatorTest' …
drishti-console/.venv/bin/python -m unittest -v tools/lake/test_s3_simulators.py      # moto: Delta and lake publish
```

**Not yet proven on AWS:** no table has been written to a real S3 bucket or committed through a real Glue catalog. First step of any production plan: `check-conditional-writes.sh`, then `check-glue.sh` against a development bucket (PROPOSED). Fixed: `lake publish --storage-option AWS_ENDPOINT_URL=…` checks the files on that endpoint, as the commit does.

# The evidence: Delta = Iceberg, to the cent



| Test | Same lines, Delta lake and Iceberg lake, one connector switched | Equal |
|---|---|---|
| `VarFormatEqualityTest` | VaR, ES, component and incremental VaR, worst scenarios, limits; every scope incl. the limit letters; USD and EUR; intraday file batches applied live; official records | to the cent |
| `ExposureFormatEqualityTest` | EE, PFE, EPE, profiles, contributions; official records | to the cent |
| `LakeAppendsTest` | the Kafka PV stream's window: one Iceberg append (on Delta, as before) | applied live |
| `IcebergParityTest`, `IcebergVectorsTest` | documents, vectors (both engines), identity, inventory against the Delta connector | bit for bit |

| On the demo data | Delta | Iceberg | Label |
|---|---|---|---|
| `VAR BANK`, 2026-09-30, USD | 112,041,646 | 112,041,646 | MEASURED (demo) |
| The risk domain moved (`--store risk=iceberg`) | 14 tables, 955,124 rows | the same, in 579 s | MEASURED (demo) |
| `LL:LL-0003`, `XPO BANK`, official records | answered | answered from Iceberg | MEASURED (demo) |

**Demo data, not bank scale.** The benchmark on both formats is **PLANNED**.

# What 2.3 million positions put in the lake each day



| Per business date | Arithmetic | Result | Basis |
|---|---|---|---|
| **P&L vectors, float64 (in the lake)** | 2.3M × 509 × 8 B | 9.37 GB[1] | arithmetic |
| **... float32 (the hot copy)** | 2.3M × 509 × 4 B | 4.68 GB | arithmetic |
| **... as Parquet (≈ 0.75 × raw)** | 9.37 GB × 0.75 | ≈ 7.0 GB[2] | ESTIMATED |
| **... a year; 7 years kept** | × 252; × 7 | ≈ 1.77 TB; ≈ 12.4 TB | ESTIMATED |
| **Partitions; files at 512–250 MB** | 1 (business_date); 7.02 GB ÷ 512–250 MB | 1; ≈ 14–28 files[3] | PROPOSED |
| **A cold read of the date** | 7.02 GB ÷ 1 MB row groups | ≈ 7,000 GETs[4] | arithmetic |
| **Portfolio definitions** | 15,000 rows a date | small | arithmetic |
| **Every portfolio's vector, if stored** | 15k × 509 × 8 B | ≈ 61 MB per currency[5] | PROPOSED |
| **Kafka micro-batches** | 5-minute window | ≤ 12 commits an hour | arithmetic |
| **Intraday records on the topic** | 10,000 updates × 4–8 KB | 40–80 MB a day | guide |

**The risk lake is small next to the documents**

≈ 12.4 TB of VaR vectors over 7 years, against ≈ 200 TB of documents (slide 86). Exposure paths are the large table (slide 52).

**Position grain**

A system may send one row per position: rows = positions, so 2.3M rows a date whatever the trade count. 15,000 portfolios are far under the 100,000-row read limit.

[1] 2.3M: bank scale (product owner); 509: the scenario set [2] × 0.75 Parquet ratio (VAR_GUIDE.md 27.4); 252 dates a year, 1,764 in 7 [3] ÷ 512–250 MB files (VAR_GUIDE.md 29.2) [4] ÷ 1 MB row groups (LAKE_ROW_GROUP_KB) [5] 15,000 portfolios: bank scale Full derivations: slides 121–123.

# Seven years at bank scale: Delta against Iceberg



| | Delta (risk domain) | Iceberg (risk domain) |
|---|---|---|
| **How 7 years are kept** | log and removed files kept 2,557 days; checkpoints | keep every snapshot 7 years (`--expire-hours` ≥ 61,368[2]) |
| **Data kept, 2.3M positions** | 1,764 dates × 7.02 GB ≈ **12.4 TB**[1] of vector Parquet (+ replaced files of restated days) | the same ≈ **12.4 TB**: the data is format-neutral |
| **Versions kept (full runs)** | 252 a year × 7 = **1,764** commits in the log | **1,764** snapshots listed in every metadata JSON |
| **Intraday micro-batch commits** | one small JSON per window per table | one snapshot per window per table (ASSUMPTION: the count depends on the window) |
| **Cost per commit** | a small JSON; replay from the last checkpoint | the metadata JSON rewritten with every snapshot kept |
| **Measured** | 7,200 files list in 84 ms from a checkpoint | 1,800 daily commits: 93 ms a commit, start 160 ms → 33 ms after regrouping |
| **With intraday micro-batches** | more small JSON files; checkpoints absorb them | more snapshots in every metadata file: compact intraday into the day's run |
| **Guard against a short setting** | `lake maintain` raises it (`retention-floor`) | `--floor-days 2557`; official tables: max snapshot age 7 years |

**The data volume is format-neutral; the history is not. PROPOSED for an Iceberg risk lake:** keep full-run snapshots for seven years and compact intraday appends into the day's run (the content store keeps the official captures); the 7-year floor is built; measure the metadata JSON at 2,000+ snapshots (with the planned benchmark) before committing.

[1] 9.37 GB × 0.75 (Parquet against raw, VAR_GUIDE.md 27.4: 'about 70–80 % of raw') = 7.02 GB a date; × 252 business dates = 1.77 TB a year; × 1,764 dates (7 years) = 12.4 TB; ESTIMATED [2] 2,557 days (7 years) × 24 h = 61,368 h

# Measured on the documents: a million trades a day, seven years



| Question (1,000,000 trades a day) | Measured |
|---|---|
| type-ahead `TRD CLY-40834` | 29 ms |
| open a trade, first time / cached | 137–157 ms / 48 ms |
| open a trade on a past business day | 288 ms |
| a search over the day, exact over 1,000,000 | 134–322 ms |
| desk P&L of all desks / another desk | 327 ms / 48 ms |
| impact of a netting set (143,982 trades) | 716–881 ms |
| a search on another business day, first / again | 6.6 s / 306 ms |
| server heap in use | 2–3.5 GB |

| Seven years: 1,800 days, 7,200 files | Measured |
|---|---|
| start-up incl. file list and newest ids | 729 ms |
| a trade on 2019-12-05 | 22 ms |
| a trade on 2022-06-15 | 13 ms |
| the file list re-read | 84 ms |
| an old day's columns | 13 ms |

MEASURED

Nothing in the read paths grows with history: a read concerns one business day, the file list comes from the last checkpoint, caches hold days by memory. A day in 2019 waits once for its id map (about a second).

# Reading one VaR date at bank scale: lake, hot file, memory

### The lake (S3, Parquet)

ESTIMATED

≈ 7.0 GB a date; a cold whole-bank pass ≈ 18–70 s on one server[1]

### The hot file (local disk)

ESTIMATED

≈ 5.5 GB a date[2]; reopened in ≈ 0.5 s; 60 dates ≈ 328 GB

### Memory (page cache + heap)

ESTIMATED

heap ≈ 1.8 GB a date[3]; whole bank ≈ 78 ms resident, ≈ 1.6 s cold from NVMe

× 23 = scaled from the measured 100,000-trade points (VAR_GUIDE.md 27.3); a node is size-independent. The 200K benchmark (measured on one workstation) keeps 2.3M extrapolated; VaR sharding (slide 65) is the step for a date larger than one server.

| Measured at 100,000 trades | Time | × 23 |
|---|---|---|
| **first load of a date** | 0.8–1.2 s | ≈ 18–28 s |
| **reopen from its hot file** | 23 ms | ≈ 0.53 s |
| **a node's VaR and ES** | 0.48 ms | ≈ 0.5 ms |
| **the whole bank, float32** | 3.4 ms | ≈ 78 ms |

**On ECS**

`DRISHTI_VAR_HOT_DIR` and `DRISHTI_EXPOSURE_HOT_DIR` on fast local disk (NVMe), as the guides say: the default `<data dir>` is on EFS. Hot files are rebuildable from the lake, so they meet the NVMe exception of slide 22.

[1] the guide's 10–40 s per 4 GB × 7.02 ÷ 4 [2] 237.7 MB × 23 (measured at 100,000) [3] 80 MB × 23; 4.68 GB ÷ 60 GB/s; ÷ 3 GB/s NVMe (assumed) Full derivations: slides 121–123.

# The parallel Java loader: JSON Lines into Delta or Iceberg



### The pipeline (drishti-lakeload)

- read: a file in byte ranges, ranged GETs on `s3://`; `.gz` / `.zst` streamed
- parse: streaming Jackson, the vector straight to `double[]`
- sort and spill: runs to `memory.spill-dir`, bounded by `memory.budget-mb`
- write: Parquet files of `file-size-mb`; ONE commit per business date

### The layout it writes

- sorted by `LEGAL_ENTITY`, `DESK`, `id`: files a shard owner can skip
- a file cut where a desk changes; a load manifest of each file's key range
- the vector column BYTE_STREAM_SPLIT + ZSTD; Delta through Kernel, Iceberg by replacePartitions
- folder or `s3://` in and out; credentials from the AWS chain only

```
drishti.py lake build --engine java --kind scenario-pv \
  --from s3://<landing>/<date>/pv/ --lake s3://<lake>/lake \
  --loader-set memory.spill-dir=/mnt/nvme/spill
```

Equal to the Python path on Delta and Iceberg, row for row and vector for vector (`PythonPathEqualityTest`). The Python path stays the default; `--engine auto --java-above-mb N` picks the loader for large loads.

| Load | Time | Basis |
|---|---|---|
| **10K positions × 510, Delta, warm** | 0.19–0.29 s | MEASURED |
| **the same into Iceberg, warm** | 0.25–0.27 s | MEASURED |
| **2.3M, 32 vCPU, 64 GB, NVMe spill** | 1–2 min[1] | ESTIMATED |
| **2.3M, Fargate 16 vCPU, 120 GB** | ≈ 2 min | ESTIMATED |
| **2.3M, 16 vCPU, 32 GB, gp3 spill** | 4–5 min | ESTIMATED |

PARALLEL_LOADER.md section 11: 2.3M not yet measured

[1] PARALLEL_LOADER.md §11's estimate from about 92–150 µs of CPU a position (scaled from the 10,000-position ThroughputTest) plus spill I/O; to be confirmed by the planned benchmark

# VaR sharded across servers, and running it on ECS Fargate



### How it works

- each date cut into shards of `LEGAL_ENTITY`, `DESK` ranges (a desk never split)
- two owners per shard by rendezvous hashing; another zone with `zone-tag`
- an owner reads only its ranges from S3: Delta files skipped by the log's statistics, Iceberg's by their bounds
- every member answers every scope: nodes from aggregates, the rest by partial vectors

```
# config/packs/market-risk/pack.yaml
sharding:
  enabled: true
  key: [LEGAL_ENTITY, DESK]
  kinds: [scenario-pv, pnl-vector]
  target-shard-mb: 2048
  replicas: 2
```

### On ECS Fargate: scale in safely

- stop hook: `drishti.py var shards --drain` (or `POST /api/v1/var/shards/drain`)
- wait for `GET /api/v1/var/shards` → `"safeToStop": true`
- `stopTimeout` ≥ the largest shard's build + `grace-seconds`

### When it cannot answer exactly, it says so

- no live owner of a shard: `503 DRS-5215`, never a partial VaR
- another data version: `409 DRS-5216`; a bad cut: `422 DRS-5217`
- nodes (VAR BANK, desks) still answer from their aggregates

Equal to one server **to the cent** on three members, USD and EUR, Delta and Iceberg, intraday included (`VarShardClusterTest`). Off by default. Drain and rolling upgrades on ECS: chapter 6.

# The demo lake: 85 Delta tables from committed JSON Lines



| What | Value |
|---|---|
| **Tables** | **85**, about 3.4 GB |
| **Business days** | 60, ending today's business date |
| **Built in** | 943 s (about 15 minutes), measured |
| **From** | committed `data/testdata/…/*.jsonl.gz` (about 138 MB) |
| **Trades** | 15,000 on the last day in 3 tables (ENDUR, CALYPSO, MUREX) |
| **Risk** | VaR inputs, limit letters, scenario PVs, FX scenarios, exposure paths for 74 netting sets |

```
drishti.py lake demo                          # all 85 tables
build_demo_lakes.py --store risk=iceberg   # risk on Iceberg
build_demo_lakes.py --names config/demo/names.yaml
```

- **The pattern:** `data/testdata/<domain>/<kind>/*.jsonl.gz` → `build_demo_lakes` → Delta → the pack's Delta connector
- tested with the demo connector **off**: a build test fails any shipped kind without a Delta connector
- the same commands build a lake on EFS for a proof of concept

Domains: trading, reference, legal, collateral, credit, **risk**, market, macro, news, civic, us-equities, gleif, funds. **Not a production ETL:** in production your own loads end with the data-load signal (DATA_LOADS.md), and on AWS the loader task does the work (slide 71).

# Public-data packs: real data on the same lake pattern



**us-equities**

- 479 companies with SEC filings
- filings, facts, index members, SIC
- prices internal only, off

**gleif**

- ≈ 9,500 legal entities
- 540 ownership groups, 132 countries
- LEI, BIC, ISIN mappings

**funds**

- 200 largest US funds and ETFs
- N-PORT holdings, N-CEN facts
- index weights from SEC data

**refresh script**

- `tools/publicdata/<pack>.py refresh`

→

**committed snapshot**

- `snapshot-<date>.jsonl.gz`

→

**lake demo**

- `<pack>/<kind>` every demo day

→

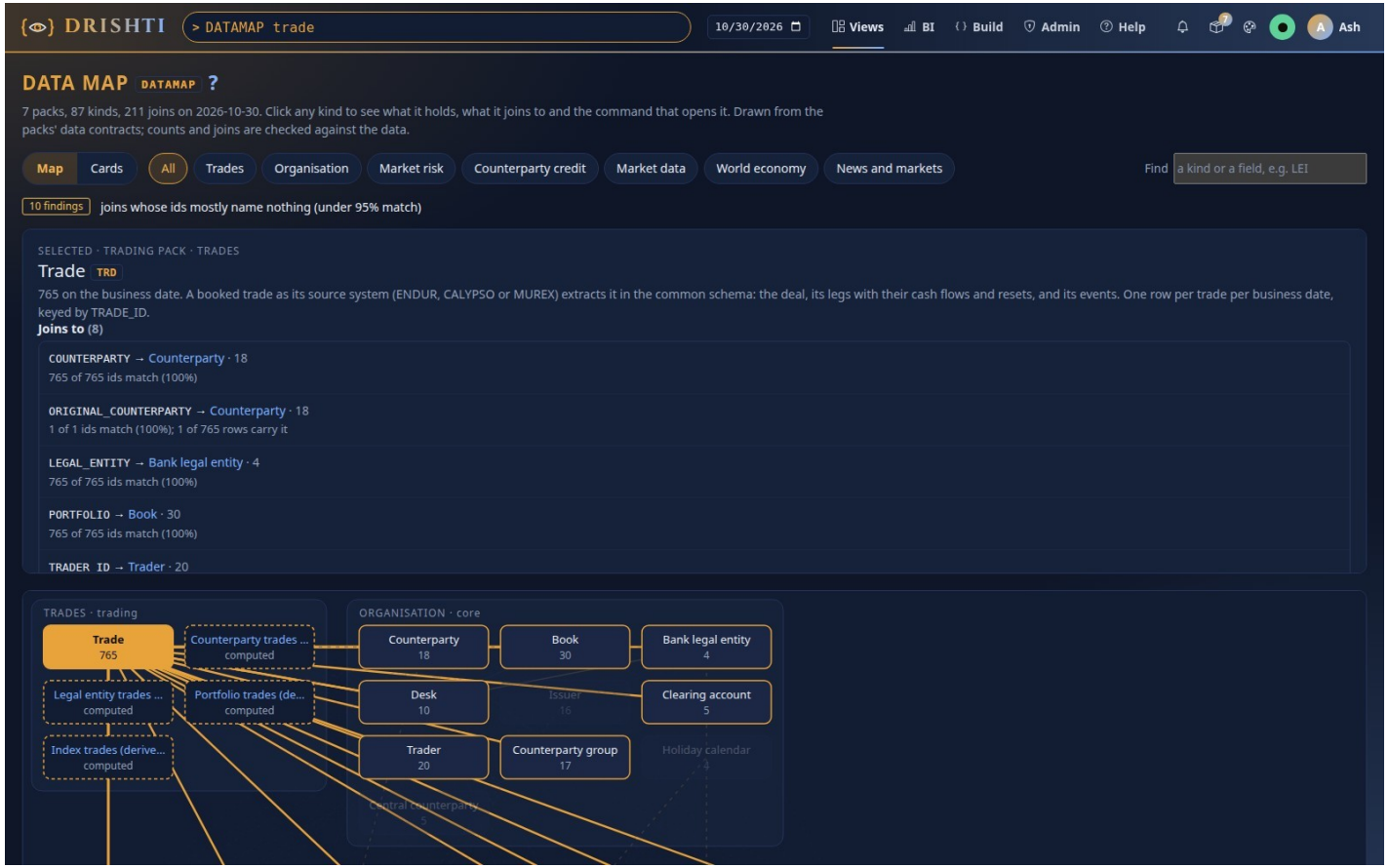
**Delta connector**

- `us-equities-store`, `gleif-store` …

- One standalone refresh script per pack (`status`, `refresh`, `--dry-run`, `--offline`), `.sh` and `.ps1` wrappers
- Internal-only sources (IMF, Yahoo, news) are refused in committed folders without `--internal-ok`
- Names in the type-ahead from promoted columns: `USEQ Ford` finds `F` (`layout.<kind>.type-ahead`)

# DATAMAP: every kind, count and join, checked against the lake






`DATAMAP trade`: the selection across the top (Trade and its joins, each checked against the lake), the map at full width under it (QUICKSTART packs, 2026-10-30).

- kinds from the packs' contracts; computed kinds (VaR, exposure) dashed
- joins from `links:`, `link:` and `graph.fields`, **each checked against the lake**: under 95 % match is a finding
- counts per business date, cached 15 minutes; read from the day's columns, never documents

**On the demo lake**

Measured 2026-10-27, 10 packs: 96 kinds, 226 joins, 176 checked in about 1.5 s; 11 findings to fix in the contracts or the data.

# Operating it

06



# Setup at a glance: steps 0 to 11, Delta and Iceberg

### 0 Prerequisites

**Delta:** VPC, endpoints, EFS, KMS, ECS, ALB, SNS, secrets
**Iceberg:** + a Glue endpoint if Glue

### 1 Settings

**Delta:** `deploy/aws/env` from env.example
**Iceberg:** `ICEBERG_TABLES_FILE`, `ICEBERG_PREFIX`

### 2 Buckets

**Delta:** `setup-buckets.sh`
**Iceberg:** unchanged

### 3 IAM

**Delta:** `setup-iam.sh`
**Iceberg:** `iceberg/setup-iam.sh`, `iam/*.json`, Glue

### 4 Prove commits safe

**Delta:** `check-conditional-writes.sh`
**Iceberg:** the same check

### 5 Storage tiers

**Delta:** `setup-lifecycle.sh`
**Iceberg:** `iceberg/setup-lifecycle.sh`: `data/` only

### 6 EFS access points

**Delta:** `ecs/setup-efs.sh`
**Iceberg:** unchanged

### 7 Cloud Map seeds

**Delta:** `ecs/setup-cloud-map.sh` (twice)
**Iceberg:** unchanged

### 8 Images

**Delta:** docker build ×3, push to ECR
**Iceberg:** the server image carries `IcebergLoader`

### 9 Tasks, services

**Delta:** `ecs/register.sh`
**Iceberg:** + `DRISHTI_ICEBERG_ROOT`

### 10 First load

**Delta:** `load-day.sh --date …`
**Iceberg:** `iceberg/load-day.sh`; `check-glue.sh`

### 11 Alarms and report

**Delta:** `setup-alarms.sh`, `report.sh`
**Iceberg:** `iceberg/` versions: snapshots

Every script: bash and the AWS CLI v2, `deploy/aws/env` (git-ignored), `--help` and `--dry-run`, safe to run again; dry-run tested, not yet run in a real account. Step by step: appendix, slides 96 to 107.

# The daily load on AWS: Delta's four steps, Iceberg's one

## Delta (built): build locally, then publish

**aws s3 sync**

- landing → EFS scratch

↓

**lake build**

- scratch Delta table

↓

**aws s3 sync**

- data files → S3

↓

**lake publish**

- one commit, conditional PUT

## Iceberg (built, `iceberg/load-day.sh`): the loader writes S3

**IcebergLoader, engine native**

- reads the landing dumps (stdin or a file)
- sorts each day by id, spills to EFS or ephemeral disk
- writes Parquet and manifests to `s3://…/iceberg/<domain>/<table>/`
- commits: conditional PUT of `vN+1.metadata.json`, then `version-hint.text`

```
deploy/aws/iceberg/load-day.sh --date 2026-10-20
  # per table in ICEBERG_TABLES_FILE, under its lock:
  # landing dumps → lake build --format iceberg \
  #   --lake s3://LAKE_BUCKET/iceberg
  # metrics: LoadSucceeded, LoadSeconds
```

**Fewer moving parts on Iceberg:** no scratch table and no second sync; the loader holds the table's lock only while it writes. Keep the Delta scripts' rules: one table per lock, `--dry-run`, exit codes, metrics to `Drishti/Lake`.

# The data-load signal



**lake publish**
- one commit on S3

→

**POST /api/v1/packs/{pack}/loads**
- per date, a `loads:write` token

→

**the server**
- refresh, verify, alert (DATA_LOADS.md)

→

**risk engines**
- VaR: reload + official capture
- exposure: reload

- With `DRISHTI_SIGNAL_URL` (a node's Cloud Map name) and `DRISHTI_TOKEN_SECRET_ID` (an API token with `loads:write`, read into a 0600 file)
- Without it, servers see the new table version at their next refresh (`refresh-seconds`, 10)
- Either way readers switch at the commit; the signal makes it immediate and runs the load's expectations and alerts
- **Risk kinds:** a `pnl-vector`, `scenario-pv`, `scenario-set`, `risk-hierarchy` or `portfolio` signal reloads the VaR date and captures it officially; an intraday batch is applied on top; an `exposure-path` signal reloads exposure
- **Iceberg:** `lake publish` sends it for Delta; for Iceberg a script sends it (`build_demo_lakes.py --server` does so locally), else the servers see the new snapshot at the next refresh (10 s)

Signals are safe in bursts: reads queue at most `max-queued-reads` (4,096) and a read whose caller has gone leaves the queue.

# Backfill and restatement, Delta and Iceberg



```
deploy/aws/delta-lake/backfill.sh --from 2026-01-02 --to 2026-10-16 --skip-missing          # weekdays; dates without dumps skipped
deploy/aws/delta-lake/backfill.sh --from 2025-01-02 --to 2025-12-31 --from-dir /opt/drishti/data/history
```

| A source re-sent | Delta (`delta-lake/`) | Iceberg (`iceberg/`) |
|---|---|---|
| **some whole days** | `backfill.sh --dates 2026-10-14,2026-10-15` | load the days again: each replaces its partition in one commit |
| **one product type of a day** | `load-day.sh --date 2026-10-15 --tables trading/trade_endur --mode overwrite-partitions` | the whole day again (product type is not a partition) |
| **years of history** | `backfill.sh --from … --to …`, oldest first; stops at the first failing date and prints the command that resumes | `backfill.sh --from … --to …`, oldest first; stops and says how to resume |
| **old days to drop** | `lake maintain` retention | `lake maintain` retention (metadata only), 7-year floor for risk |

- Re-running a date replaces it: a backfill can always be run again; a large one runs several loader tasks side by side on different tables (`--tables`), the per-table lock keeps two apart
- Readers switch at the commit, never a mixture; data files of an old restated day tier by their date prefix at the next lifecycle run (the rule is by date, not object age)

**Restatement:** *known at* before it still shows the day as it was, until vacuum (`LAKE_VACUUM_HOURS`; 7 years for risk) or snapshot expiry. A restated VaR or exposure date gets the next `RESULT_VERSION`; the earlier one is kept.

**Guarded:** removing more than half of a table's rows fails, removing nothing, unless `--force-drop`; a row dated after tomorrow in the business zone is not loaded and the load ends with an error naming it.

# What the nightly maintenance runs, Delta and Iceberg



Nightly (`MAINTAIN_SCHEDULE`, Tuesday to Saturday 02:30) as the maintenance task, per table under the table's lock; then setup-lifecycle.sh.

```
drishti.py lake maintain --config <lake-maintenance.s3.yaml rendered> --once --force-foreign --tables trading/trade_endur
deploy/aws/iceberg/maintain.sh     # IcebergMaintenance per table; every option: appendix
```

| Step | Delta (`maintain.sh`, lake-maintenance.s3.yaml) | Iceberg (`iceberg/maintain.sh` → `IcebergMaintenance`) |
|---|---|---|
| **Retention** | `keep-business-days` (`LAKE_KEEP_BUSINESS_DAYS`, 1800): dates older than that, one commit; refuses to drop more than `max-drop-share` (0.5) | `--keep-days 1800`: days older than seven years, metadata only; more than half the rows needs `--force-drop` |
| **Checkpoint · manifests** | `checkpoint: true`; removes commit files past `delta.logRetentionDuration` | `--rewrite-manifests`: one manifest per month; planning reads a few dozen |
| **Vacuum · expiry, orphans** | `vacuum-hours` (`LAKE_VACUUM_HOURS`, 168): files no version uses; S3 keeps them 14 more days as noncurrent | `--expire-hours 168`: keeps the metadata JSON small, bounds *known at*; `--remove-orphans`: files no snapshot names, 72 h at least, a dry run lists them |
| **Compaction · relayout** | `compact: false` (slide 76) | `--relayout`: re-sort days another writer appended out of order; fold delete files into data |
| **Risk floor** | 7 years: 1,827 days, 61,368 h; `risk/`, `risk-results/` never below it (`retention-floor`) | `--floor-days 2557`: no date dropped, no snapshot expired inside 7 years |

Iceberg, measured: connector start on 1,800 daily commits 160 ms → 33 ms after manifests and expiry; 93 ms a commit; drift check 0.7 s, keep + rewrite + manifests + expiry 2.9 s (10,000 trades).

> `--force-foreign`: the S3 Delta tables are Drishti's (lake publish wrote them), so the script overrides the ownership rule and marks the runs forced. A managed compaction must keep the id sort (PROPOSED rule).

# Storage tiers by business date, for both formats

| 0 – 90 days | 90 days – 2 years | 2 years – kept days | beyond |
|---|---|---|---|
| S3 Standard | Standard-IA (`TIER_IA_AFTER_DAYS`) | Glacier Instant Retrieval | removed by `lake maintain` / `IcebergMaintenance` → noncurrent expiry |

business date age → read latency: milliseconds in every tier

```
{"ID": "tier-trading-trade_endur-2026-03", "Status": "Enabled",
 "Filter": {"And": {"ObjectSizeGreaterThan": 131072,
   "Prefix": "lake/trading/trade_endur/business_date=2026-03"}},
 "Transitions": [
   {"Date": "2026-06-29T00:00:00Z", "StorageClass": "STANDARD_IA"},
   {"Date": "2028-03-30T00:00:00Z", "StorageClass": "GLACIER_IR"}]}
```

- **Never** Glacier Flexible Retrieval or Deep Archive: a restore of hours before a read
- **Never** an S3 expiry of current table files, in either format: it deletes files the log or a manifest still names; S3 keeps noncurrent versions 14 days

- One rule per table per business month, with dates (age is wrong for restated days); three months ahead; rewritten nightly
- **Iceberg:** the same rule, dates and tiers on `iceberg/<domain>/<table>/data/business_date=YYYY-MM`
- `_delta_log/` and `metadata/` never match; objects under 128 KB stay
- S3 allows 1,000 rules; the script refuses more (≈ 30 tables over 7 years[2]); each format's script keeps the other's rules

**The risk lake:** `risk/` and `risk-results/` keep 7 years of dates and time travel; old months sit in Glacier IR, still read in milliseconds, so any official figure can be recomputed years later.

**Costs to know:** Standard-IA keeps an object at least 30 days, Glacier IR 90 days; a restatement of a tiered day pays the rest of that minimum. Reads: Standard-IA ≈ USD 0.01/GB[1], Glacier IR ≈ USD 0.03/GB.

[1] S3 retrieval list prices (one Region) as quoted in AWS_DELTA_LAKE.md 9.4; minimum durations: AWS's rules [2] 1,000 rules ÷ about 32 rules a table (5 whole years in Glacier IR + 27 months moving) ≈ 31

# Compaction on S3: relayout by id, compaction off by default



- Each day arrives laid out: files of `file-rows`, sorted by id, one commit ⇒ the S3 tables need no compaction
- A plain Delta compaction would merge files out of id order (single-entity reads would touch many row groups)
- `lake maintain` re-sorts (**relayout**) instead of compacting where the pack declares a layout
- So `compact: false` stays the shipped S3 setting; a day that needs re-laying is restated (slide 115)

**Relayout, not compaction**

`lake maintain` finds the layout of the connector whose table it is, so `trade_endur`, `trade_calypso` and `trade_murex` are re-laid out by id, not compacted out of order; relayout keeps `business_date` and `PRODUCT_TYPE` and checks each.

**Turn compaction on for S3 after a relayout test there**

PROPOSED

- the shipped S3 setting keeps `compact: false` (`lake-maintenance.s3.yaml`); turn it on only after this test
- run `lake maintain --dry-run` with `compact: true` on one S3 table, check id order per row group, then enable it nightly

# Admin → Lakes: the same engine in the console



| Step | What it does | Setting |
|---|---|---|
| **Retention** | deletes business dates older than the window, counted from today; refuses to drop more than half a table | `keep-business-days` |
| **Compaction** | merges small files, or re-lays a table in its pack's layout (dates that drifted only) | `target-file-mb` (128) |
| **Checkpoint** | readers replay a short log | `checkpoint` (on) |
| **Vacuum** | removes files no version uses, older than the window | `vacuum-hours` (168) |

- **Dry run** first: files and size before → after, dates retention would drop, what vacuum would delete
- **Ownership rule:** only tables `lake build` built; a foreign table is read-only
- Schedules at a time or after every build; history kept in `maintenance-history.jsonl`
- **Iceberg tables** (`format: iceberg`, or auto under a local root) go to `IcebergMaintenance` under the same rules; the tables list has a Format column

### On AWS: one scheduler

EventBridge runs `maintain.sh` nightly against `s3://` with `--force-foreign`.

Admin → Lakes maintains the lakes listed in `lakes.roots` and the file `<data>/lakes/lake-maintenance.yaml`; dry runs and history work the same.

Run schedules in **one** place: set `DRISHTI_LAKE_MAINTENANCE_SCHEDULES=false` on the console when EventBridge or the daemon runs them.

The risk floor applies in every place it runs.

# What never scans the whole lake

| Request | Reads | Grows with history? |
|---|---|---|
| **Open one entity on a date** | the snapshot or planned layout (in memory, refreshed every 10 s), the day's id map, one row group (≈ 1 MB) | no |
| **History of one entity over N days** | the days whose id map holds it; one read per day, eight at once | with N, not with the lake |
| **Type-ahead** | the newest day's ids and promoted names in memory | no |
| **Search, pick list, derived kind, impact, DATAMAP count** | that day's column set (id + promoted, never documents); files and row groups skipped by bounds | no |
| **A VaR or exposure date** | one streamed pass over the date's vectors, then the hot file | no |
| **Refresh (Iceberg)** | `version-hint.text` (path) or `loadTable` with ETag (REST); a changed snapshot's manifests | with live manifests: regroup monthly |
| **Load, restatement, intraday batch** | one day's files and one commit (Iceberg rewrites the metadata JSON) | no (Iceberg: metadata JSON grows with snapshots kept) |
| **Retention** | a delete by partition (the log or metadata only), one commit | no |
| **Vacuum (weekly is enough)** | a listing of the table's prefix | with the file count |
| **Questions over all of history** | not interactive: Athena, Trino or Spark on the same tables | — |

**The one Iceberg-specific growth:** each commit rewrites a metadata JSON listing every snapshot kept: expire snapshots (seven-year risk tables: slide 61) and regroup manifests monthly.

# History over the lake: TRD <id> 30



```
TRD 1000015 30                    # the trade over its last 30 business days with data
TRD 1000015 3M DIFF               # three months, each day's changes
GET /api/v1/history/trade/1000015?days=30&knownAt=2026-10-15T18:00:00Z    # as the lake stood then
```

**days()**

- the snapshot's `business_date=` partitions whose id map holds the id

**one read a day**

- id map → one row group; eight days at once

**mask, then diff**

- each day masked for the caller; changed fields

**answer, cached**

- by the kind's data version

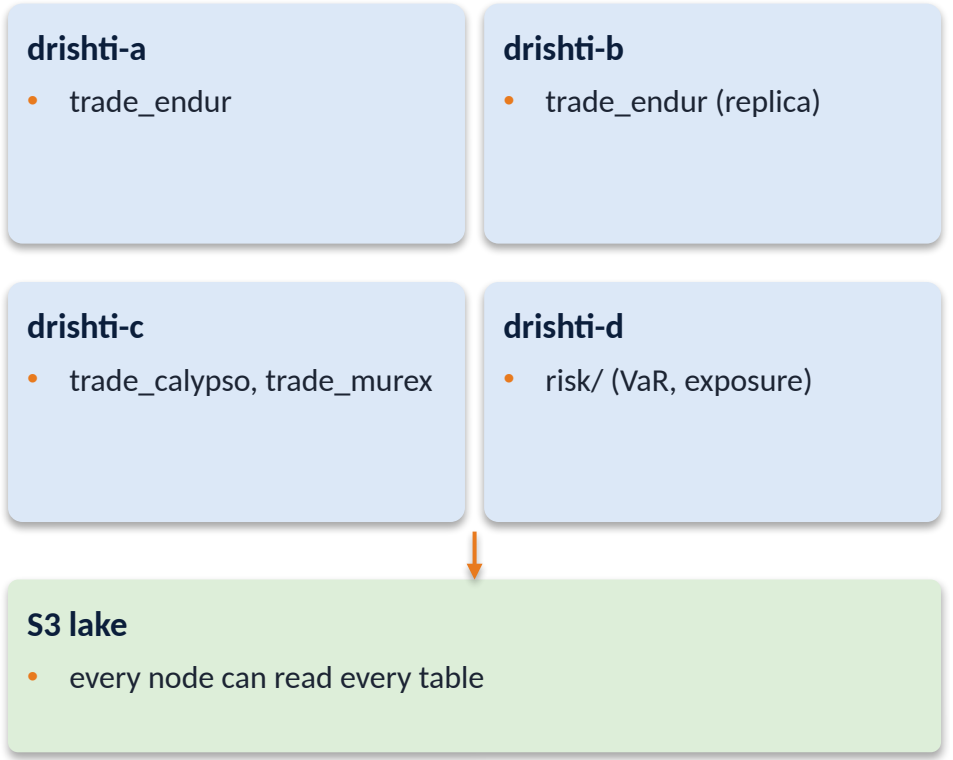
- A weekend, a holiday or a day the entity was not loaded costs nothing
- `knownAt` is Delta time travel: the day as the table stood then
- Across Drishti*net: one call to the peer that holds the data, masked on both sides
- Works the same on Iceberg (equality test) and on every connector (the contract's default)

**Measured on 1,800 days**

a trade on 2019-12-05 in 22 ms; the history reads only the days that hold the entity, so 7 years of lake cost no more than the days asked.

# Spreading the work over Drishti*net

- **Shards:** nodes whose connectors read different tables of a kind; a search asks every one; a read goes to the holder
- **Replicas:** nodes on the same table; a forwarded read goes to the same node for the same entity (hash of its id)
- Every node can read every table from S3: a failed node costs speed, never answers
- Peer data over HTTP/2, one connection per peer; gossip on its own port; history across nodes
- **VaR sharded:** each owner reads only its legal-entity and desk ranges of a date (slide 65)
- **Iceberg tables too:** the plugin reports `iceberg:<table-uuid>`, so two nodes on one Iceberg table are replicas (a search asks one), also after a copy read through `relocate`




**DESIGNED (SUPER_SCALE S5, S6):**

ownership of (table, day) slices of the documents' tables on the ring, so N servers on one table hold N × one server's working set. (The VaR date is already sharded by legal entity and desk, partial vectors added in shard order: built.)

# Zero downtime on ECS: readiness, drain, rolling upgrades



**Drain**
- POST /api/v1/admin/node/drain

→ **Safe to stop**
- safeToStop: true

→ **Update the service**
- the new task definition

→ **Services stable**
- aws ecs wait

→ **Ready**
- /actuator/health/readiness 200

**Order:** consoles → BI and data-serving → coordinators → shard owners, one zone at a time. Any failure stops the run, names the node and says how to resume (`--from`).

| Readiness | HTTP | When |
|---|---|---|
| `UP` | 200 | joined, newest date open, shards ready |
| `DOWN` | 503 | joining, a date opening, a shard not ready |
| `OUT_OF_SERVICE` | 503 | drained before a stop |

**On ECS**
- one service per node: `maximumPercent` 100, `minimumHealthyPercent` 0
- the task's health check is liveness: a node waiting for a date is never restarted
- the ALB target group reads the console's `/readyz`; owners are never behind it
- `stopTimeout` ≥ the largest shard's build + `grace-seconds`

```
export DRISHTI_DRAIN_TOKEN=drk_...      # an administrator's token with the node:drain scope
tools/rolling-upgrade.sh --inventory prod-ecs.yaml --version <version> --dry-run   # every step printed, nothing called
```

The inventory is `deploy/rolling.example.yaml` with `platform: ecs`: the cluster, the Region and each node's service and task definition.

# Alerts to Prometheus: the platform rules, in PromQL

| Alert | Expression (abridged) | Severity |
|---|---|---|
| `DrishtiServerDown` | up{job="drishti"} == 0, 2m | critical |
| `DrishtiCacheNearLimit` | cache bytes / limit > 0.95, 1h | warning |
| `DrishtiHeapAfterGcHigh` | live data / max > 0.85, 10m | warning |
| `DrishtiDirectMemoryHigh` | direct / max > 0.8 | critical |
| `DrishtiDiskFreeUnderGuard` | free < guard: new dates refused | critical |
| `DrishtiSourceDown` | drishti_source_up == 0, 5m | critical |
| `DrishtiFeedsNeedAttention` | late, missing or failed loads | warning |
| `DrishtiViewsSlow` | page p99 above 150 ms, 5m | warning |
| `DrishtiMemberSuspect` | a member silent, 2m | warning |

```
# prometheus.yml
rule_files:
  - drishti-alerts.yaml
scrape_configs:
  - job_name: drishti
    metrics_path: /actuator/prometheus
```

### With or without Prometheus

- each server evaluates the same rules itself
- an alert that opens is logged, counted and sent to every administrator's inbox
- Admin → Health lists what is open now
- the CIO lens reads the same metric names

One file to load: `deploy/prometheus/drishti-alerts.yaml`; route `critical` and `warning` in Alertmanager. The lake's own CloudWatch alarms (step 11) stay as they are.

# The numbers to watch: report.sh



```
bucket acme-drishti-prod-lake (CloudWatch daily storage metrics):
  StandardStorage                         10412.3 GB[1]
  StandardIAStorage                       46120.8 GB
  GlacierInstantRetrievalStorage         143998.1 GB
  objects                        672310

table                      version  checkpoint  behind   dates       newest   files GB(newest)    <32MB
trading/trade_endur           4512        4510       2    1764   2026-10-20     128      41.37        0
```

- `report.sh` is scheduled daily (`REPORT_SCHEDULE`), prints these and sends them to CloudWatch; numbers illustrative for ≈ 200 TB
- **behind** = commits since the last checkpoint: intraday risk tables add commits all day; checkpoints every 10 keep it short
- **<32MB** = small files of the newest day: should be 0 for the documents; intraday batches are small by nature (maintenance compacts them)

[1] illustrative output for a lake of about 200 TB (AWS_DELTA_LAKE.md 11.3): 10.4 + 46.1 + 144.0 TB = 200.5 TB, the 10 / 46 / 144 TB tier split of 12.3

# The numbers to watch: healthy values and what to do



| Number | Healthy | If not |
|---|---|---|
| **behind (commits since the checkpoint)** | under 10 | a checkpoint failed: run maintain.sh; alarm log-long above 50 |
| **files of the newest day, and <32MB** | ≈ rows ÷ file-rows per partition; 0 small | tiny dumps or partitions too fine (slide 14) |
| **live files per table** | under ≈ 100,000 | split the table (by source, or by year: S2, DESIGNED) |
| **S3 5xx** | rare | 503 Slow Down on one prefix: spread with partitions; alarm lake-s3-5xx |
| **Admin → Health p50Ms, p99Ms, cold reads** | p95 < 300 ms newest days, < 1 s any day | footers and id maps not warm, or EFS latency |
| **Admin → Caches: the one memory budget** | under its cap | raise the heap (8 GB per million entities a day) or shard |
| **Admin → Health `gossipPort`, `peerProtocol`** | a port, `http2` | 0: gossip on the server port (a firewall; slide 103) |
| **LoadSeconds** | well inside the window | one loader task per table, or larger loader tasks |

# The numbers to watch, for an Iceberg table

```
{"name": "trade-calypso", "health": "UP", "engine": "native",
 "cache": {"tables": 1, "snapshots": 1, "idMaps": 2, "columnSets": 2, "documents": 2,
           "partitions": 0, "daysWithDeletes": 0, "ids": 10000}}
```

| Number | Healthy | If not | Status |
|---|---|---|---|
| `daysWithDeletes` | 0 for Drishti's tables | single reads slow: `IcebergMaintenance --relayout` | |
| **Health:** `not laid out as the pack declares` | absent | `--columns trade:path,…` or reload | |
| `DEGRADED: … names files that are not there` | absent | a moved table: set the `relocate` pair Health gives | |
| **snapshots kept (`IcebergSnapshots`, alarm at 500)** | bounded by `--expire-hours` | commits and refreshes slow: expire | |
| **live manifests per table** | a few dozen | planning slow: `--rewrite-manifests` | **PROPOSED** metric |
| **single-writer warning in the log** | absent | the store ignores `If-None-Match`: one writer per table | |

# What 200 TB is (seven years of Parquet)



| | Value |
|---|---|
| **Per business day (252 a year)** | about 113 GB[1] of Parquet; about 100 million documents at about 1.1 KB each compressed (Snappy) |
| **Files per day** | about 350 (330 MB each, 250,000 rows) over all tables |
| **Live files after seven years** | about 620,000[2] over all tables; three trade tables ≈ 200,000 each: above 100,000, so tables by year (S2) or more tables matter before year seven |
| **Commits** | one per table per day, plus maintenance: about 1,500 a year per table[3] (intraday risk tables more) |
| **The risk lake at 2.3M positions** | ≈ 7 GB of VaR vectors (Parquet) a date, ≈ 12.4 TB over 7 years (slide 60) |

ESTIMATED Estimates, to be replaced by the measurements of SUPER_SCALE step S8.

[1] 100 million documents × 1.1 KB (Snappy) = 110 GB, rounded up in AWS_DELTA_LAKE.md 12.1; 113 GB × 252 × 7 = 199 TB [2] 350 files a day × 1,764 dates = 617,400; 350 × 330 MB = 115.5 GB a day [3] the doc's estimate: 252 load commits plus nightly maintenance commits; not broken down

# The deployment



| Part | Size | Count |
|---|---|---|
| **Server tasks** | 4 vCPU and 12 GB for the server (8 GB heap) in a 6 vCPU, 16 GB task with its console; 16 vCPU, 64 GB at SUPER_SCALE volumes | 3 to 6, sharded by table |
| **Loader task** | 4 vCPU, 30 GB, 21 GB ephemeral | one per table at load time |
| **Maintenance and report tasks** | the same image and size | nightly, minutes |
| **EFS** | each node's state and caches: 50–300 GB; loader scratch: one day of one table | one file system |
| **A node reading the risk lake** | heap ≈ 1.8 GB per open VaR date[1] at 2.3M; hot files ≈ 5.5 GB a date on local NVMe (EC2) | ≥ 1; whole dates each |

[1] 80 MB × 23 and 237.7 MB × 23, from the measured 100,000-trade VarTierBenchmark

# Monthly cost: Delta, and what changes with Iceberg



| Item | Basis | About (Delta) | With Iceberg |
|---|---|---|---|
| **S3 storage, 200 TB tiered** | 10 TB Standard (USD 23/TB), 46 TB Standard-IA (12.5), 144 TB Glacier IR (4) | USD 1,400[1] | the same tiers; zstd may store less (not measured side by side) |
| **S3 storage if all were Standard** | 200 TB × 23 | USD 4,600 | the same |
| **S3 requests** | 250 reads a second, two GETs each, USD 0.40 per million; loads and lists | USD 500–700[2] | about the same; a few more small objects per commit |
| **Retrievals from IA and Glacier IR** | month-ends, audits, history | tens to a few hundred | the same |
| **Versioning overhead** | 14 days of replaced and vacuumed files | a few % of storage | the same |
| **Catalog** | — | none | none (path-based); Glue or S3 Tables: their charges |
| **Server nodes** | 3 tasks of 6 vCPU, 16 GB on Fargate, always on | USD 650–750[3] | the same tasks |
| **Loader, maintenance** | about an hour a day | USD 50–150 | fewer steps; managed optimizers charged apart |
| **EFS** | 1 TB Standard, Elastic throughput | USD 300 + throughput[4] | the same (the same disk caches) |
| **CloudWatch, Secrets Manager, Cloud Map, endpoints, load balancer** | | USD 200–400 | + a Glue interface endpoint if Glue |

The same 200 TB on EFS: USD 60,000 a month[5] in Standard; about USD 3,200 in IA plus USD 0.03 per GB read. **Either format, the same order of magnitude:** storage dominates; the choice is about capability and operations, not cost.

[1] tier split × S3 list prices [2] 250 reads/s × 2 GETs × a month × USD 0.40/M [3] 3 Fargate tasks × list prices × 730 h [4] 1,000 GB × USD 0.30 [5] 200,000 GB × USD 0.30 (EFS Standard) Full derivations: slides 121–123.

# Disaster recovery, Delta and Iceberg



| Event | Delta | Iceberg |
|---|---|---|
| **A wrong commit** | time travel: the previous version is readable until vacuum (7 years for risk); restate the days from the landing bucket (30 days) | *known at* reads the snapshot before it; restate the days from the landing bucket |
| **Files deleted by mistake (a wrong vacuum, a script on the wrong prefix)** | S3 versioning: restore noncurrent versions within `NONCURRENT_KEEP_DAYS` (14) (list-object-versions, copy-object) | the same |
| **A table moved or copied (a laptop, another bucket)** | — | `relocate: '<old> -> <new>'`, the pair Health gives (the library engine ignores it); the table identity stays |
| **A Region lost** | S3 Cross-Region Replication with RTC, EFS and ECR replication; fail over once the backlog is empty | CRR; read the replica with `relocate: 's3://prod/… -> s3://dr/…'` (PROPOSED: test it); repoint a REST catalog's metadata location too |
| **A commit arrived before its files (replication)** | that day fails (Health DEGRADED) until the file arrives | the same; a `version-hint.text` ahead of its metadata reads NoSuchTable until it arrives |
| **Node state lost** | AWS Backup of EFS; caches and hot files rebuild from the lake | the same |
| **Cluster state (content store)** | s3 backend: the bucket's versioning and replication; filesystem on EFS: AWS Backup and EFS replication | the same |

**The Iceberg-specific DR rule:** absolute paths make a copy a different table until `relocate` maps it; with a REST catalog, the catalog's metadata location must be repointed in the recovery Region too.

# What comes next

07



# Designed: the super-scale scale-out of the lake

| Item | What it adds | Why it matters at scale |
|---|---|---|
| **S1** | Range maps from the log's per-file id statistics; Parquet Bloom filters on id | find an id without building a day's id map |
| **S2** | Tables by year (`root: …/{year}`), closing a year | a table's checkpoint stops growing with seven years of live files |
| **S3** | A loader fleet writing slices of one table at once; a day register | loads of very large days finish in the window |
| **S4** | Products (link indexes, group totals, rollups, id lists) written with the data | aggregates and links without scanning |
| **S5, S6** | Each server owns a share of (table, day) slices on the ring | N servers hold N × one server's working set |
| **S7, S8** | Warm tier of memory-mapped column files; benchmark at scale in AWS | hot days from memory; measured numbers |

Built, so not in this table: VaR sharded across servers (each owner holds only its ranges of a date; partial vectors added in shard order) and Drishti*net shards by table.

DESIGNED

# Proposed by this deck: not yet in the docs



| # | Proposal | Why |
|---|---|---|
| 1 | Set node.yaml `gossip-port` to a fixed number on ECS and open it in the `drishti-node` security group | `auto` changes the port at each start; behind a security group gossip falls back to 18480 |
| 2 | Kafka windows written to an EFS lake, then copied to S3 by `lake publish --mode append` after each window | the stream appends to a lake path, not `s3://` |
| 3 | Re-enable `compact` for the S3 trade tables after a relayout test on S3 | `lake maintain` now lays out per-source tables |
| 4 | `exposure-path` partitioned by date and run, sorted by netting set (as EXPOSURE_GUIDE.md proposes) | a netting set in one streaming pass |
| 5 | A published table of node and portfolio vectors per currency (from the VaR deck) | BI and peers read sums, not 2.3M rows |
| 6 | Change data feed on the risk input tables; Z-order or liquid clustering by book and portfolio (VaR deck) | followers see intraday deltas; a desk's read skips files |
| 7 | Load the risk lake with the parallel Java loader (`lake build --engine java`), not the Python path | its files are sorted by legal entity and desk, BSS + ZSTD: a node is a run of rows |
| 8 | Iceberg: path-based tables first; Glue's REST endpoint when Athena or Spark must share a table; S3 Tables after a proof | the same commit safety as Delta with no extra service (slide 37) |
| 9 | Iceberg format v3 reading (deletion vectors, new types), named branches and tags; a *Make a pack* button for `iceberg-external` | listed as later in the Iceberg docs |

PROPOSED

# Limits and what comes next



| Limit | Status | Source |
|---|---|---|
| Not yet run in a real AWS account (steps 0–11, runbooks); no Iceberg table written to a real bucket | to do | AWS_DELTA_LAKE.md §15 |
| The Glue and S3 Tables live runs (code and `check-glue.sh` written) | NOT RUN | AWS_DEPLOYMENT.md §22 |
| The benchmark: 200K positions, one server against three sharded, both formats; 2.3M extrapolated; demo figures are demo only | PLANNED | IMPLEMENTATION_PLAN.md |
| Delta in-place schema evolution; Delta ↔ Iceberg conversion of any table; Unity Catalog | PLANNED | IMPLEMENTATION_PLAN.md |
| Kafka windows are appended to a lake path (local or EFS), not `s3://` | limit; to S3 PROPOSED | VAR_GUIDE.md §8.4 |
| More than 100,000 hierarchy nodes or portfolios from a connector that cannot list a date whole (15,000 portfolios fit) | limit | VAR_GUIDE.md §20 |
| Tables by year, range maps, slice ownership (S1–S8: scale-out) | DESIGNED | SUPER_SCALE.md |
| `lake build` sorts a partition only when it fits a batch | limit | AWS_DELTA_LAKE.md §7.2 |
| A bucket without conditional writes: one writer per table | limit | AWS_DELTA_LAKE.md §8.2 |
| Iceberg spec gaps, refused by name: format v3 (deletion vectors, new types), encrypted tables, ORC and Avro data files, branches and tags | limit | DRISHTI_ICEBERG.md §5 |
| The native Iceberg engine reaches local disk and S3 only; Azure, GCS, HDFS need `engine: library` (Hadoop) | limit | ICEBERG_CONNECTOR.md §17 |
| Doc fix: ICEBERG_CONNECTOR.md §4 says path tables on S3 'need a single writer' (true only of the library engine) | to do | ICEBERG_CONNECTOR.md §4 |

# Decisions to take, with this deck's recommendation



| # | Decision or action | Recommendation |
|---|---|---|
| 1 | Disk caches and VaR/exposure hot files on local NVMe (breaks the identical-path rule) or EFS | Hot files on NVMe from the start (the guides ask for fast local disk); connector caches on EFS until p99Ms shows EFS latency |
| 2 | First real run in a development AWS account | Before any production commitment: steps 0–11 and the runbooks; `check-conditional-writes.sh`, `check-glue.sh` (Glue and `--s3-tables`), one Iceberg load and read; record the outputs in the guide |
| 3 | Official results straight to `s3://`; how Kafka windows reach S3 | Official: `s3://` root, add `s3:PutObject` to the server role. Kafka: EFS lake, then `lake publish --mode append`, 7-year retention (PROPOSED) |
| 4 | A fixed gossip port on ECS | A number in node.yaml and a security-group rule node to node (PROPOSED) |
| 5 | Compaction on S3 | Re-enable after a relayout test on one S3 table (PROPOSED) |
| 6 | Tables by year (S2); range maps, Bloom filters (S1); slice ownership (S5, S6); VaR sharding on | When live files near 100,000 per table, or one server's working set or one VaR date becomes the limit |
| 7 | Which format for the **risk domain** (vectors, official results) | Delta by default; Iceberg where the bank standardises on it (equal to the cent); decide on a bank-scale benchmark's measured figures |
| 8 | Which format for **trades and reference data** | Either, per source system; Iceberg where the bank's pipelines already write it |
| 9 | The **catalog** on AWS for Iceberg | Path-based first; Glue's REST endpoint when Athena or Spark must share the tables; S3 Tables only after a proof |
| 10 | Seven-year history on Iceberg | The floor is built; keep full-run snapshots, compact intraday; measure metadata at 2,000+ snapshots |

# Appendix



# Step 0: prerequisites (the platform team's templates)



- A VPC with private subnets in three Availability Zones
- A gateway VPC endpoint for S3 on their route tables (lake traffic never leaves the VPC; the bucket policy requires it)
- Interface endpoints for ECR, CloudWatch Logs, Secrets Manager and STS
- An EFS file system: Regional, encrypted, Elastic throughput, a mount target per subnet
- A KMS key for the lake; its key policy allows the roles of step 3
- An ECS cluster
- An Application Load Balancer with a target group for the consoles on port 17480
- An SNS topic for alarms
- The four secrets of AWS_DEPLOYMENT.md §11.3 (token secret, session secret, cluster key, admin seed)

**Who builds these?**

The platform team, with its own CloudFormation or Terraform templates. The Drishti scripts take their ids (subnets, file system, key ARN, cluster, target group, topic) from `deploy/aws/env`.

**Run the scripts from**

a workstation or admin host with AWS CLI v2, the repository, Java 21 and `drishti-console/.venv`.

Content store on S3 (optional): its bucket needs conditional writes too (step 4).

# Step 1: the settings file



```
cp deploy/aws/env.example deploy/aws/env        # git-ignored; holds account ids and names, never a secret
$EDITOR deploy/aws/env                          # replace every <placeholder>
cp deploy/aws/delta-lake/tables.example deploy/aws/delta-lake/tables.conf   # your tables; set LAKE_TABLES_FILE
```

**You should see:** nothing yet. A script run with a placeholder left names it: `setup-buckets.sh: LAKE_KMS_KEY_ARN is still a placeholder (<arn:aws:kms:…>) in deploy/aws/env`

```
trading/trade_endur     trade   config/packs/trading   trade-endur     {date}/endur/          -      PRODUCT_TYPE
```

| Field | Example | Meaning |
|---|---|---|
| **folder** | `trading/trade_endur` | the table's folder under the lake prefix (domain / table) |
| **kind** | `trade` | the Drishti kind |
| **pack** | `config/packs/trading` | the pack whose layout (key, promoted and vector columns) is used |
| **connector** | `trade-endur` | the pack's connector that reads this table |
| **landing prefix** | `{date}/endur/` | where the JSON Lines dumps are; `{date}` is the business date |
| **business-date field** | `-` | the field holding the date, or - for the day being loaded |
| **partition column** | `PRODUCT_TYPE` | the second partition key |

# Step 2: the buckets



```
deploy/aws/delta-lake/setup-buckets.sh --dry-run
deploy/aws/delta-lake/setup-buckets.sh
+ aws s3api create-bucket --bucket acme-drishti-prod-lake --object-ownership BucketOwnerEnforced \
      --create-bucket-configuration LocationConstraint=eu-west-2
+ aws s3api put-bucket-versioning --bucket acme-drishti-prod-lake --versioning-configuration Status=Enabled
+ aws s3api put-bucket-encryption --bucket acme-drishti-prod-lake --server-side-encryption-configuration file://…
+ aws s3api put-bucket-policy --bucket acme-drishti-prod-lake --policy file://…/policy-….json
setup-buckets.sh: done: acme-drishti-prod-lake, acme-drishti-prod-landing, acme-drishti-prod-logs
```

- **Three buckets:** `-lake` (every Delta table, the risk lake included), `-landing` (JSON Lines dumps, 30 days), `-logs` (access logs, 400 days)
- **On the lake:** versioning on; SSE-KMS with S3 Bucket Keys; Block Public Access; bucket-owner-enforced ownership; TLS only; VPC endpoint only (plus `drishti-<env>-admin*` roles); access logs; request metrics on `lake/`
- **Safety net:** noncurrent versions expire after 14 days; incomplete multipart uploads abort after 2 days
- **Idempotent:** run again, it says `acme-drishti-prod-lake exists: kept, settings applied again`
- `--dry-run` prints each changing call as `+ aws …` and each read as `? aws …`; none is made

# Step 3: who may do what (IAM)



```
deploy/aws/delta-lake/setup-iam.sh --dry-run
deploy/aws/delta-lake/setup-iam.sh
```

| Role | Policy (deploy/aws/delta-lake/iam/) | May |
|---|---|---|
| `drishti-<env>-server-task` | `server-policy.json` | read the lake prefix (GET, LIST), decrypt; mount EFS through access points |
| `drishti-<env>-loader-task` | `loader-policy.json` | read the landing bucket; write data files and commits; never delete (an explicit deny) |
| `drishti-<env>-maintenance-task` | `maintenance-policy.json` | checkpoints, retention and vacuum: the only role that deletes; the lifecycle rules |
| `drishti-<env>-execution` | AmazonECSTaskExecutionRolePolicy + `drishti/<env>/*` secrets | ECS pulls images, writes logs, injects secrets |
| `drishti-<env>-scheduler` | inline | EventBridge Scheduler runs the lake tasks |

**You should see** `setup-iam.sh: done: three policies, five roles (…)`. The shipped server policy only reads; for official results to S3 add `s3:PutObject` on their prefix, no delete (AWS_DEPLOYMENT.md §22; slide 49).

# Step 4: prove that commits are safe



```
deploy/aws/delta-lake/check-conditional-writes.sh
check-conditional-writes.sh: ok: the second create was refused (412 PreconditionFailed): concurrent commits are safe
```

- **What it does:** creates a test object with `If-None-Match: *`, then tries again; S3 must refuse the second with 412
- **Why it matters:** a Delta commit is the creation of `_delta_log/<version>.json`; two writers must never both create one version
- **Any other answer: stop.** A store or proxy that drops the header cannot guarantee safe concurrent commits; then only one writer per table (the scripts' per-table locks give that if nothing else writes)
- **No DynamoDB lock table:** deltalake 1.x commits with conditional PUT

**The same guarantee, used three times**

- **Delta commits:** `lake publish`, `lake maintain` (deltalake 1.x)
- **Iceberg commits:** drishti-lakeio's conditional PUT, probed at start (a store without it: single writer)
- **Content store on S3:** create `If-None-Match: *`, update `If-Match: <ETag>`; a lost race is `409 DRS-5057`

# Step 5: storage tiers by business date



```
deploy/aws/delta-lake/setup-lifecycle.sh --print | head -20     # the rules, no AWS call
deploy/aws/delta-lake/setup-lifecycle.sh
```

**You should see** `setup-lifecycle.sh: 129 rules[1] for acme-drishti-prod-lake` (three tables from 2019).

- **By date, not by age:** age is the time since an object was written, wrong for a restated or backfilled day; one rule per table per business month, with transition dates
- Standard → Standard-IA when the month's last day is 90 days old (`TIER_IA_AFTER_DAYS`); → Glacier Instant Retrieval at two years (`TIER_GLACIER_IR_AFTER_DAYS`)
- Never Glacier Flexible Retrieval or Deep Archive (hours to restore); never an S3 expiry of current lake objects
- `_delta_log/` never matches: commits and checkpoints stay in Standard; objects under 128 KB stay where they are
- Rules exist three months ahead; maintain.sh rewrites them nightly; S3 allows 1,000 rules and the script refuses more
- **Risk tables:** 7 years kept, old months in Glacier IR, still read in milliseconds (official numbers stay reproducible)

[1] 3 tables × (5 whole years 2019–2023 already in Glacier IR + 37 months 2024-01 to 2027-01, three months ahead) + 3 base rules = 129

# Step 6: EFS access points, one per node per path



```
deploy/aws/ecs/setup-efs.sh --dry-run
deploy/aws/ecs/setup-efs.sh
```

**You should see** one access point id per line (`fsap-0a1b…`) and `setup-efs.sh: 14 access points on fs-… for three nodes`.

| Owner | Access points | Root (identical path inside the container) |
|---|---|---|
| **each node (drishti-a, -b, -c)** | data, config, sutras | `/drishti/<env>/<node>/opt/drishti/<path>` → `/opt/drishti/<path>` |
| **consoles** | data, config, sutras | `/drishti/<env>/consoles/opt/drishti/<path>` |
| **loader** | data | `/drishti/<env>/loader/opt/drishti/data` (scratch lakes, table locks) |
| **maintenance** | data | `/drishti/<env>/maintenance/opt/drishti/data` |

- Every access point runs as uid/gid 10001 (the image's user); 3 × 3 + 3 + 1 + 1 = 14
- Idempotent: the access point's name is its client token, so running again returns the same ids
- The rule from AWS_DEPLOYMENT.md: every path a container writes maps to the identical path on EFS

# Step 7: Cloud Map seeds and the Drishti*net transport



```
deploy/aws/ecs/setup-cloud-map.sh        # once for the namespace (it takes a minute) …
deploy/aws/ecs/setup-cloud-map.sh        # … and again for one service per node
```

- ECS registers each node's task under `drishti-a.drishti.local` with the task's own address
- Servers advertise `http://<task IP>:18480`, never loopback (refused)
- A new node finds the cluster through its seed's Cloud Map name, then learns the rest by gossip
- The loader's data-load signal (`DRISHTI_SIGNAL_URL`) also uses a node's Cloud Map name

**Gossip on its own port**

- node.yaml `gossip-port`: auto · a number · off
- busy nodes never suspected: 12 s saturated, no suspicion

**HTTP/2 between nodes**

- routing.yaml `peer-protocol: http2` (h2c), one connection per peer
- 1 connection instead of 227; p99 117 ms vs 370 ms

**On ECS: a fixed gossip port** PROPOSED

- set `gossip-port` to a number (e.g. 18490) on every node
- open it node to node in the `drishti-node` security group

# Step 8: the images



```
./mvnw -q clean package -DskipTests
docker build -f deploy/docker/ecs/server.Dockerfile  -t "$ECR_REGISTRY/drishti-server-ecs:<version>" .
docker build -f deploy/docker/ecs/console.Dockerfile -t "$ECR_REGISTRY/drishti-console-ecs:<version>" .
docker build -f deploy/docker/ecs/loader.Dockerfile  -t "$ECR_REGISTRY/drishti-loader-ecs:<version>" .  # carries env
docker push …                                         # each of the three, after aws ecr get-login-password
```

| Image | Built from | Entry point checks / role |
|---|---|---|
| `drishti-server-ecs` | `deploy/docker/ecs/server.Dockerfile` | server-entrypoint.sh: EFS mounted and writable, real IPv4 (never loopback), first-start node.yaml and peers; health check |
| `drishti-console-ecs` | `deploy/docker/ecs/console.Dockerfile` | console-entrypoint.sh: the console, a separate container from the server |
| `drishti-loader-ecs` | `deploy/docker/ecs/loader.Dockerfile` | loader-entrypoint.sh: EFS mounted and writable; runs the lake scripts; copies deploy/aws/ with your env (no secret) |

- Loader tasks read the same settings: `DRISHTI_AWS_ENV=/opt/drishti/app/deploy/aws/env`; rebuild the loader image when they change
- The loader image carries the Python lake tools (`lake build`, `lake publish`, `lake maintain`, `drishti.py convert`)
- The build context is the repository root; nothing is built or run on a developer laptop by the scripts

# Step 9: task definitions, services and schedules



```
deploy/aws/ecs/register.sh --dry-run
deploy/aws/ecs/register.sh                          # --nodes --lake-jobs
deploy/aws/ecs/register.sh --consoles 4             # optional: a console service of its own
```

**You should see:** a task definition and a service per node (`drishti-prod-drishti-a`, …), the loader, maintenance and report task definitions, three schedules (`drishti-prod-load-day`, `-maintain`, `-report`), then `register.sh: done`.

- The node task sets `DRISHTI_DELTA_ROOT=s3://<lake bucket>/lake`: every shipped Delta connector reads the lake in S3 with no edit, **the risk lake's** `risk-store`, `exposure-store`, `var-results` and `exposure-results` included
- The official-results publishers default to the same root and write it on `s3://` (Delta, or Iceberg); the task role needs `s3:PutObject` there (slide 49)
- Templates: `deploy/aws/ecs/templates/` → node-task, node-service, console-task, console-service, lake-task, schedule-target
- `--cache nvme` (EC2 launch type) mounts the host's `/mnt/nvme/drishti/<node>` at `/opt/drishti/nvme` (slide 22)
- Schedules: `LOAD_SCHEDULE` (22:30 Mon–Fri), `MAINTAIN_SCHEDULE` (Tue–Sat 02:30), `REPORT_SCHEDULE` (daily)

# Step 10: the first load



```
deploy/aws/delta-lake/load-day.sh --dry-run --date 2026-10-20 --tables trading/trade_endur
deploy/aws/delta-lake/load-day.sh --date 2026-10-20 --tables trading/trade_endur
```

**You should see** the steps of the daily load (slide 113), then:

```
lake publish: ok exit=0 published <n> file(s), <bytes> bytes, dates 2026-10-20 to s3://…/trade_endur version=0 (new table)
load-day.sh: business date 2026-10-20: exit 0
```

On ECS the same runs as the loader task:

```
aws ecs run-task --task-definition drishti-prod-loader \
  --overrides '{"containerOverrides":[{"name":"loader","command":["deploy/aws/delta-lake/load-day.sh","--date","2026-10-20"]}]}' …
```

**In the console**

`TRD <id>` on 2026-10-20 opens the trade; **Admin → Health** shows `trade-endur UP (engine: native)`; `DATAMAP trade` counts the day's trades and checks their joins against the lake.

**The dumps are JSON Lines**

One object a line, `.jsonl.gz`. JSON arrays still load; a CSV export is converted first: `drishti.py convert trades.csv` (CSV input is deprecated).

# Step 11: alarms and the report



```
deploy/aws/delta-lake/setup-alarms.sh; deploy/aws/delta-lake/report.sh
deploy/aws/iceberg/setup-alarms.sh;    deploy/aws/iceberg/report.sh
```

**You should see** `setup-alarms.sh: 17 alarms put`[1] for three Delta tables (four per table, two for the bucket and EFS, one per node), and the report (slide 83).

| Alarm | Fires when |
|---|---|
| **`drishti-<env>-<table>-load-failed`** | a load of the table failed (nothing of it was published) |
| **`drishti-<env>-<table>-not-loaded`** | no successful load for three days running (a weekend is two) |
| **`drishti-<env>-<table>-load-slow`** | a load took longer than `ALARM_PUBLISH_DEADLINE_HOURS` |
| **`drishti-<env>-<table>-log-long` (Delta)** | readers replay more than `ALARM_COMMITS_SINCE_CHECKPOINT` commits |
| **snapshots (Iceberg)** | more than `ALARM_ICEBERG_SNAPSHOTS` (500) snapshots kept |
| **`drishti-<env>-lake-s3-5xx`** | more than `ALARM_S3_5XX_PER_5MIN` server errors in 5 minutes, twice in 15 |
| **`drishti-<env>-efs-io-limit`** | EFS PercentIOLimit above 80 % |
| **`drishti-<env>-<node>-down`** | a node's service has no running task for 5 minutes |

All notify `ALARM_SNS_TOPIC_ARN`; `report.sh` is scheduled daily (`REPORT_SCHEDULE`) and sends its numbers to CloudWatch (Iceberg: `IcebergSnapshots`, `NewestBusinessDate`). Heap, Drishti*net member dead and backup failed come with the platform's agent; Drishti*net health is on **Admin → Health** and **Admin → Net**.

[1] 4 per table × 3 tables + 2 (bucket, EFS) + 1 per node × 3 = 17

# IAM, KMS and the network for Iceberg



| Role | Delta lake (built policies) | Iceberg lake (`iceberg/iam/*.json`) |
|---|---|---|
| **server task** | GET, LIST on `lake/`; `kms:Decrypt` | the same on `iceberg/`; with Glue: `glue:GetCatalog`, `GetDatabase`, `GetTable` (read) |
| **loader task** | PUT data files and commits; **never delete** (explicit deny) | PUT data, manifests, `vN.metadata.json`, `version-hint.text`; the deny can stay (old metadata removal is best effort); no `--keep-days` in the loader; with Glue: `glue:UpdateTable`, `CreateTable` |
| **maintenance task** | retention, checkpoints, vacuum: the only role that deletes | `--keep-days`, `--expire-hours`, `--relayout`: deletes data, manifests, old metadata |
| **KMS** | SSE-KMS bucket default, S3 Bucket Keys | unchanged: the native writer sends no SSE header; bucket default applies. Iceberg table encryption: refused |

## Network

- S3 gateway endpoint, bucket policy allowing only it (as Delta): unchanged
- Path-based tables need nothing else
- Glue catalog: an interface endpoint for Glue, or traffic to the Glue endpoint through the proxy settings
- Vended credentials: STS endpoint (already among the Delta prerequisites)

**The proxy and TLS settings carry over:** `proxy.*` and `tls.*` apply to the REST catalog's client and to the object store (ICEBERG_CONNECTOR.md: Security; Outbound proxy).
Private CA for the catalog's own HTTPS: import into the JVM truststore.

# Columns of every table



| Column | Type | Content |
|---|---|---|
| `id` | string | the entity's id (the pack's key: `TRADE_ID`, `VECTOR_ID`, …) |
| `doc` | string | the whole JSON document as the source sent it (a vector replaced by a marker) |
| `business_date` | date | partition |
| **partition columns** | string | `PRODUCT_TYPE` |
| **promoted columns** | string or double | copies of the fields searches, lists, links and BI read: `layout.<kind>.columns` |
| **vector columns** | `list<double>` (or `list<float>`) | `layout.<kind>.vectors`: `PNL`, `SCENARIO_PV`, `MTM`; one column, the length is data |

- Choosing what to promote is a pack decision (DELTA_CONNECTOR.md §4); `layout.<kind>.type-ahead` adds promoted names to the type-ahead (`USEQ Ford` finds `F`)
- Adding a promoted column changes the table's columns: `lake publish` refuses a day whose columns differ → runbook (slide 115); in place: planned, not built
- Vectors are never stored twice: the document keeps `{"$vector": "PNL"}` and a fetched document gets the vector back

# Columns and table properties



| Column (field id) | Type | Content |
|---|---|---|
| `id` (1) | string, required | the entity's id: `MX-20000001` |
| `doc` (2) | string | the whole JSON document, exactly what a view shows |
| `business_date` (3) | date, required | the identity partition |
| promoted (`TRADE_ID`, `PORTFOLIO`, ...) | string / double | the pack's `layout.<kind>.columns`; `LEGS.CURRENCY` → `LEGS__CURRENCY` |

| Property | Value | Why |
|---|---|---|
| partition spec / sort order | `identity(business_date)` / `id ASC` | a day is a unit; engines that honour the sort order keep it |
| `format-version` | 2 | position and equality deletes |
| `write.parquet.row-group-size-bytes` | 1,048,576 | one read decodes about 3 MB of documents (sized compressed) |
| `write.parquet.compression-codec` | zstd | about 0.7 KB per 6.5 KB trade (measured) |
| `write.metadata.metrics.column.doc` / `.id` | none / full | no bounds of documents; exact id bounds per file |
| `write.metadata.previous-versions-max` | 50 (delete after commit) | old metadata files removed |
| `commit.retry.num-retries` | 10 | concurrent day commits retry |
| `drishti.layout.file-rows` | 250,000 | rows per file; maintenance re-cuts drifted days |

# What of the Iceberg specification is covered



| Area | Covered | Refused or later |
|---|---|---|
| **Table metadata** | every v1/v2 field; unknown fields kept when it writes the next version | format version 3 (refused) |
| **Types** | boolean, int, long, float, double, decimal, date, time, timestamp(tz), string, uuid, fixed, binary, struct, list, map | v3 types: timestamp_ns, variant, geometry |
| **Schema evolution** | by field id: add, drop, rename, reorder, int→long, float→double, decimal widening; name mapping | — |
| **Partitioning** | identity, bucket (Murmur3), truncate, year, month, day, hour, void; several specs (evolution) | — |
| **Manifests** | lists and manifests (Avro), data and delete manifests, inherited ids and sequence numbers, bounds | — |
| **Deletes** | position and equality deletes, by the sequence-number rules | v3 deletion vectors (Puffin): later |
| **Snapshots** | current snapshot, snapshot log (time travel by *known at*), the `main` ref | named branches and tags: later |
| **Data files, encryption** | Parquet | ORC, Avro data files; encrypted tables (refused) |
| **Catalogs** | path-based, a metadata file named directly, REST | Hive metastore, JDBC (only behind a REST catalog) |

# Maintenance as built: IcebergMaintenance



| Option | What it does (one Iceberg commit each) |
|---|---|
| `--keep-days N` | deletes business days older than the newest N on or before `--as-of`: metadata only; > 50 % of rows needs `--force-drop` |
| `--relayout` (`--force`) | rewrites only days that drifted (overlapping id ranges, oversized files, delete files, missing columns), sorted, validated against the snapshot read |
| `--columns kind:path,…` | adds promoted columns and rewrites every day, filling them from the documents |
| `--rewrite-manifests` | regroups manifests, one per month of business dates |
| `--expire-hours H` | expires snapshots older than H (168), keeps the current, deletes files only they referenced |
| `--remove-orphans --older-than-hours` | files no snapshot references (a load that died before its commit), 72 h at least, with a dry run |
| **Admin → Lakes,** `lake maintain` | an Iceberg table maintained as a Delta one: ownership rule, lock, schedules, history, a Format column |
| **The 7-year floor of the risk domains** | `--floor-days 2557` for `risk` and `risk-results`: no date dropped, no snapshot expired inside it |

**Never** apply S3 lifecycle **expiry** to a table's files, in either format: they delete files the metadata still names.

# The Delta daily load in detail, and its runbook



```
EventBridge Scheduler (LOAD_SCHEDULE, LOAD_SCHEDULE_ZONE)
  └─► ECS RunTask drishti-<env>-loader ─► loader-entrypoint.sh (EFS mounted? writable?) ─► load-day.sh
        for each table of LAKE_TABLES_FILE:
          flock  /opt/drishti/data/lake-locks/<domain>--<table>.lock   (held: exit 3)
          aws s3 sync   s3://landing/<date>/<source>/  →  scratch/in/<table>/      (JSON Lines)
          drishti.py lake build   --from scratch/in/<table> -r --kind trade --pack config/packs/trading
                                  --mode overwrite-dates --batch-rows 250000 --partition-by PRODUCT_TYPE …
          aws s3 sync   scratch/lake/trading/trade_endur/ → s3://lake/…/trade_endur/  --exclude "_delta_log/*"
          drishti.py lake publish --from scratch/lake/… --to s3://…/trade_endur --dates <date> [--signal …]
          CloudWatch Drishti/Lake: LoadSucceeded, PublishedBusinessDate, LoadSeconds
        scratch removed (always)
```

| Exit | Meaning | What to do |
|---|---|---|
| **0** | every table published | check: CloudWatch Logs `/drishti/<env>`, stream prefix loader, `LoadSucceeded` 1 per table; `report.sh --tables <table>`: newest is today; `TRD <id>` on today's date; `DATAMAP` counts the day |
| **2** | published with rejected documents (the summary says which) | the build summary names the reasons; fix the source and restate the day |
| **1** | a table failed: its S3 table is unchanged | `lake build failed` or `lake publish failed` says why; fix, then `load-day.sh --date <d> --tables <t>` |
| **3** | a table was locked | maintenance or a backfill held the table's lock; the next run loads it, or run it by hand after |

The date defaults to today in `LOAD_SCHEDULE_ZONE`: an evening load in New York is already the next day in UTC.

# A backfill and a restatement



### A backfill

1. `backfill.sh --dry-run --from … --to … --tables <t>`: check the dates and commands
2. Run it as a loader task per table (`aws ecs run-task … --overrides` with `backfill.sh …`)
3. It stops at a failing date with the resume command; fix and resume
4. `setup-lifecycle.sh` (or the next maintain.sh) moves the backfilled years to their tiers

### A restatement

1. The source re-sends the JSON Lines dumps under the landing prefix of the day
2. `backfill.sh --dates <d>` (whole day) or `load-day.sh --date <d> --tables <t> --mode overwrite-partitions`
3. Readers switch at the commit; the data-load signal (if set) refreshes the servers at once

# A slow table



| Symptom | Cause | Fix |
|---|---|---|
| **first read of a day slow, then fast** | id map not yet on disk; footers cold | normal; `warm-dates` (3) warms the newest days at start |
| **every read of a day slow** | the day is not sorted by id, or row groups are large | restate the day after fixing the order (slide 17) |
| **searches slow or partial** | a field the search uses is not promoted | promote it; publish then refuses the next day: backfill into `trade_endur_v2` and switch `table.trade` (in place: not built) |
| **the snapshot refresh slow (Health)** | many live files or a long log (intraday commits) | report.sh: behind and file counts; run maintain.sh; split the table |
| **503 Slow Down** | one prefix overloaded | partition by `PRODUCT_TYPE` (or finer); S3 scales a prefix within minutes |
| **searches read documents on one table** | deletion vectors in its files (written by Spark) | compaction rewrites the files without them |

# A restated risk date



1. The risk batch rewrites the whole date (a new `BATCH_ID`, `RESTATEMENT` + 1) as JSON Lines
2. Load it like any table: `load-day.sh --date <d> --tables risk/pnl-vector` (overwrite-dates)
3. The data-load signal reloads the date: a new hot file (new inputs' digest); the intraday layer is dropped
4. The official capture runs again: the next `RESULT_VERSION`, appended; the earlier version is kept
5. Check: `drishti.py var official` lists both versions; BI shows the latest; `publishError` empty

**If the date did not load**

`DRS-5203` (two runs or scenario sets on one date): the engine answers the date before, with a warning. Fix the batch, rewrite the date whole.

**Nothing is lost**

Time travel keeps the old inputs 7 years; the official table keeps every result version; a capture keeps its vector. "What did we report?" and "what is the corrected number?" both stay answerable.

# An upgrade: the connector files follow the pack



### A file nobody edited

- replaced in full by the pack's new template; the old one kept as `.before-upgrade-….bak`; audited `connector-template-upgrade`

### A file you edited

- gains every key the template has, at any depth; your values stay; lists keep their order

### A value that differs

- listed in Admin → Connectors → Template updates, with **Use the pack's value** per key

the stamp is in the file's text: `# drishti-template: pack=… template=…`

### Tested on the real files

`ConnectorUpgradeTest` starts a server on an earlier release's generated files, an edited `risk-store` and an edited older `var-whatif`: unedited files equal the new templates, edits stay, VaR, what-if and exposure answer.

### On ECS

Deploy the new images; at start each node brings its connector files (EFS or the content store) in step. Check **Template updates** after the first start.

Data written before a new column (the limit letters' `LIMIT_LETTER`) skips that hierarchy alone with a `DRS-5211` warning; MGMT, LEGAL and exposure answer as before (`HierarchyUpgradeWithoutLettersTest`).

# Runbooks: symptoms, causes, fixes



| Symptom | Likely cause | Fix |
|---|---|---|
| `DOWN: cannot reach …` | root, bucket or catalog namespace not reachable | check `root` / `uri`, the task role, `s3.*` |
| `DOWN: no Iceberg tables under …` | nothing loaded in that domain, or the wrong `domain` / `namespace` | load it; check the domain |
| **new data not visible** | the refresh has not run yet | wait `refresh-seconds` (10) or send the data-load signal |
| **single reads slow,** `daysWithDeletes` **> 0** | another engine's equality deletes | `IcebergMaintenance --relayout` |
| **single reads slow on another engine's table** | large row groups or unsorted days | `--relayout --force`, or that engine's sorted rewrite |
| **searches say** `partial: true` | a field the query reads is not promoted | add it to the layout, then `--columns` |
| ***known at* finds nothing** | the snapshots of that time were expired | keep more history (`--expire-hours`) |
| **log:** `… does not honour conditional writes … single-writer` | the store ignores `If-None-Match` | one writer per table, or a REST catalog |
| **loader:** `iceberg: the load failed: <why>` | the first reason in the failure's chain | fix and load the day again (idempotent) |

# deploy/ and tools/lake/: the files



```
deploy/
├── docker/              compose, server and console Dockerfiles
│   └── ecs/             server, console, loader images + entry points
├── aws/
│   ├── env.example      every setting; copy to deploy/aws/env
│   ├── lib/common.sh    --help, --dry-run, --env, templates
│   ├── delta-lake/      setup-buckets, setup-iam,
│   │                    check-conditional-writes, setup-lifecycle,
│   │                    setup-alarms, load-day, backfill, maintain,
│   │                    report, lake-maintenance.s3.yaml, iam/
│   ├── iceberg/         the same set for Iceberg
│   └── ecs/             setup-efs, setup-cloud-map, register,
│                        templates/*.json
├── grafana/             the Grafana dashboard
└── lake-maintenance.yaml   lake maintain's local example
```

```
tools/lake/
├── build.py  publish.py  maintain.py
├── demo.py   stage.py  writers.py
├── stores.py  stores_equality.py
├── s3sim.py   javaload.py (--engine java)
└── test_publish  test_maintain
    test_s3_simulators  test_vectors
    test_retention  test_names
tools/samplegen/load_rows.py
tools/convertcmd.py   drishti.py convert
```

`drishti.py lake publish` (TOOLS.md §4.11) and the S3 tests (§4.12) run in the drill; `tools/test_deploy_aws.py` runs every script with --help and --dry-run.

# Conventions every script follows



- bash with `set -euo pipefail` and the licence header
- AWS CLI v2 and nothing else; Python only through `tools/drishti.py`
- `--help` on every script; `--dry-run`: each changing call printed as `+ aws …`, each read as `? aws …`, none made
- Settings from one file: `deploy/aws/env` (from env.example; placeholders only; git-ignored), or `--env <file>`
- Safe to run again (idempotent): buckets, policies, access points and rules are kept and their settings applied again
- A placeholder left stops the script and names it
- Locks: load, backfill and maintenance take the table's flock on EFS; a held lock exits 3
- Data in is JSON Lines; a CSV source is converted first with `drishti.py convert`
- Tested: `tools/test_deploy_aws.py` (every script with --help and --dry-run against a fake settings file and an aws that fails if called; every template rendered; lifecycle rules; env git-ignored); `test_publish.py` and the S3 simulator tests run in the drill

# How the numbers were derived (1/3)



| Slide | | How it was arrived at |
|---|---|---|
| **2: Executive summary** | 1 | 10 TB × USD 23 + 46 TB × 12.5 + 144 TB × 4 a TB-month = USD 1,381; the tier split by business date and the S3 list prices (one Region) from AWS_DELTA_LAKE.md 12.3; EFS: 200,000 GB × USD 0.30 a GB-month (EFS Standard list price) = USD 60,000 |
| **2: Executive summary** | 2 | 2,300,000 positions × 509 scenarios × 8 B = 9.37 GB a date (float64; × 4 B = 4.68 GB float32); positions: the product owner's bank scale; 509: the scenario set (VAR_GUIDE.md). 9.37 GB × 0.75 (Parquet against raw, VAR_GUIDE.md 27.4: 'about 70–80 % of raw') = 7.02 GB a date; × 252 business dates = 1.77 TB a year; × 1,764 dates (7 years) = 12.4 TB; ESTIMATED |
| **6: Why S3 and not EFS: the comparison** | 1 | 10 TB × USD 23 + 46 TB × 12.5 + 144 TB × 4 a TB-month = USD 1,381; the tier split by business date and the S3 list prices (one Region) from AWS_DELTA_LAKE.md 12.3 |
| **6: Why S3 and not EFS: the comparison** | 4 | AWS_DELTA_LAKE.md §2's figure for EFS Infrequent Access; it implies USD 0.016 a GB-month over 200,000 GB; to be confirmed against the current EFS price list |
| **52: Exposure paths: the large table, and what is kept for 7 years** | 1 | trades × P paths × G grid dates × 4 B: 1,000,000 × 2,000 × 50 × 4 = 400 GB; × 5,000 × 100 = 2.0 TB; float64 doubles them; 100,000 sets × P × G × 4 B for the V matrices |
| **52: Exposure paths: the large table, and what is kept for 7 years** | 2 | 2.0 TB × 1,764 dates = 3.53 PB; kept tiers (EXPOSURE_GUIDE.md 27.3): 32 dates × 2.0 TB = 64 TB + 50 TB (a year of set matrices) + 17 TB (84 month-ends) + 0.85 TB (statistics, contributions, captures) = 132 TB; the guide says about 150 TB |

# How the numbers were derived (2/3)



| Slide | | How it was arrived at |
|---|---|---|
| **60: What 2.3 million positions put in the lake each day** | 1 | 2,300,000 positions × 509 scenarios × 8 B = 9.37 GB a date (float64; × 4 B = 4.68 GB float32); positions: the product owner's bank scale; 509: the scenario set (VAR_GUIDE.md) |
| **60: What 2.3 million positions put in the lake each day** | 2 | 9.37 GB × 0.75 (Parquet against raw, VAR_GUIDE.md 27.4: 'about 70–80 % of raw') = 7.02 GB a date; × 252 business dates = 1.77 TB a year; × 1,764 dates (7 years) = 12.4 TB; ESTIMATED |
| **60: What 2.3 million positions put in the lake each day** | 3 | 7.02 GB ÷ 0.512 GB = 13.7; ÷ 0.25 GB = 28.1 |
| **60: What 2.3 million positions put in the lake each day** | 4 | 7.02 GB ÷ 1 MB = 7,024 ranged GETs |
| **60: What 2.3 million positions put in the lake each day** | 5 | 15,000 × 509 × 8 B = 61.1 MB a date a currency |
| **63: Reading one VaR date at bank scale: lake, hot file, memory** | 1 | the guide's 10–40 s for a pass over about 4 GB of Parquet × 7.02 ÷ 4 = 18–70 s (VAR_GUIDE.md 27.4); ESTIMATED |

# How the numbers were derived (3/3)



| Slide | | How it was arrived at |
|---|---|---|
| **63: Reading one VaR date at bank scale: lake, hot file, memory** | 2 | × 23 = 2,300,000 ÷ 100,000: linear from VarTierBenchmark, measured at 100,000 trades × 5 dates on one workstation (VAR_GUIDE.md 27.3): 237.7 MB × 23 = 5.47 GB; × 60 disk-dates = 328 GB |
| **63: Reading one VaR date at bank scale: lake, hot file, memory** | 3 | 80 MB × 23 = 1.84 GB; whole bank 4.68 GB float32 ÷ 60 GB/s (measured rate) = 78 ms; ÷ 3 GB/s NVMe (assumed, the VaR deck's figure) = 1.6 s |
| **88: Monthly cost: Delta, and what changes with Iceberg** | 1 | 10 TB × USD 23 + 46 TB × 12.5 + 144 TB × 4 a TB-month = USD 1,381; the tier split by business date and the S3 list prices (one Region) from AWS_DELTA_LAKE.md 12.3 |
| **88: Monthly cost: Delta, and what changes with Iceberg** | 2 | 250 reads/s × 2 GETs × 2,592,000 s = 1.30 billion GETs × USD 0.40 a million = USD 518, plus loads and lists |
| **88: Monthly cost: Delta, and what changes with Iceberg** | 3 | 3 tasks × (6 vCPU × USD 0.04048 + 16 GB × USD 0.004445) × 730 h = USD 688 (Fargate list prices, us-east-1, Linux/x86; the Region is an assumption) |

# Glossary



| Term | Meaning |
|---|---|
| **Delta Lake** | a table format: Parquet data files plus `_delta_log/`, an ordered log of JSON commits and Parquet checkpoints |
| **Apache Iceberg** | a table format: Parquet data files plus `metadata/`: a metadata JSON, a manifest list per snapshot, manifests |
| **Snapshot / version** | the table as one commit left it; time travel reads an earlier one until expiry or vacuum |
| **Catalog** | where an engine finds a table's current metadata: path-based (`version-hint.text`) or a REST service (Glue, Polaris) |
| **Conditional PUT** | an S3 create with `If-None-Match: *`: a second create of the same key is refused (412); commits rely on it |
| ***Known at*** | a view as the data stood at an instant (`X-Drishti-Known-At`, `knownAt`), read through time travel |
| **Promoted column** | a field copied out of the document into a column so searches, lists and BI never read documents |
| **Vector column** | `list<double>`: a trade's P&L over the scenarios, its scenario PVs, or an exposure path, in one column |

| Term | Meaning |
|---|---|
| **Id map** | the per-day map from an entity's id to its file and row group; on disk, memory-mapped |
| **Hot file** | a VaR or exposure date in memory-mapped float32 form on local disk, rebuilt from the lake |
| **Landing bucket** | where the sources drop their JSON Lines dumps, by date and source, for 30 days |
| **lake build / publish / maintain** | the lake tools: JSON Lines to a table; a built day committed into S3; retention, checkpoints, vacuum, relayout |
| **IcebergLoader / IcebergMaintenance** | Drishti's Java loader and maintenance for Iceberg tables, in the server's exec jar |
| **Retention floor** | the 7 years the `risk` and `risk-results` domains always keep, whatever the settings say |
| **Drishti*net** | several Drishti servers as one: gossip, routing, replicas and shards, HTTP/2 between nodes |
| **Data-load signal** | `POST /api/v1/packs/{pack}/loads`: tells the servers a date has landed; they refresh and check it |

# Related documents



| Document | What it holds |
|---|---|
| **docs/architecture/AWS_DELTA_LAKE.md** | the Delta lake on AWS: every section, command and number of the scripts |
| **docs/architecture/AWS_DEPLOYMENT.md · SUPER_SCALE.md · SCALE_STRATEGY.md** | ECS, EFS, the identical-path rule, DR; §22: Iceberg on AWS and official results to S3; S1–S8 |
| **docs/architecture/DRISHTI_ICEBERG.md** | the native Iceberg engine: protocol coverage, catalogs, commits, two connectors |
| **docs/connectors/ICEBERG_CONNECTOR.md · ICEBERG_EXTERNAL_CONNECTOR.md** | the `iceberg` connector, loader, maintenance; `iceberg-external` |
| **docs/connectors/DELTA_CONNECTOR.md · CONNECTOR_FILES.md** | how Drishti reads Delta; layout, vectors, history; connector upgrades |
| **docs/admin/LAKE_MAINTENANCE.md · docs/guides/TOOLS.md** | lake maintain, Admin → Lakes, the 7-year floor; lake build, publish, the S3 tests |
| **docs/admin/NET.md · DRISHTI_NET.md · CONTENT_STORE.md** | Drishti*net: gossip port, HTTP/2, replicas and shards; the content store on S3 |
| **docs/guides/DATA_FORMATS.md** | JSON Lines is the standard; CSV deprecated; official results as JSONL |
| **docs/guides/VAR_GUIDE.md · EXPOSURE_GUIDE.md** | the risk lake: inputs, vectors, intraday, official results, sizes |
| **docs/architecture/VAR_SHARDING.md · PARALLEL_LOADER.md** | VaR sharded across servers, Fargate drain; the parallel Java loader |
| **docs/connectors/DEMO_DATA.md · docs/guides/PUBLIC_DATA.md** | the 85-table demo lake, mixed storage; the public-data packs |
| **Drishti-VaR-Engine.pptx · Drishti-Design-and-Evidence.pptx** | the companion decks: the VaR engine; the whole product |

Drishti दृष्टि

# Thank you

*See what your data hasn't told you yet.*

ANY DATA. ANY DOMAIN. ONE GRAMMAR.

Drishti on AWS: the lake, Delta or Iceberg

Ashutosh Sinha · October 2026